INTELLIGENT EMPLOYEE PRODUCTIVITY

TRACKING SYSTEM (PTA)

FINAL YEAR PROJECT REPORT

B.S. IN SOFTWARE ENGINEERING

By

Student	Roll Number
Taqi Rahmani	2022F-BSE-283
Arqum Faisal	2022F-BSE-254
Ibad Tahir	2022F-BSE-344
Yagome Hina	2022F-BSE-305

Supervised By

Engr. Zainab Zakir

2025-2026

DEPARTMENT OF SOFTWARE ENGINEERING

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY

KARACHI, PAKISTAN

Preface

This report presents the design, implementation and evaluation of PTA, a behavioural productivity-analytics system developed as a Final Year Project. The project began as a systematic review of AI-driven workforce monitoring and grew into a working system: five endpoint telemetry services, a central database with a server-side feature-engineering layer, three machinelearning models, a serving API, and a public dashboard.

The report distinguishes four categories of statement. **Implemented** means a feature exists in the working tree. **Deployed** means it runs on the project server and was verified there. **Measured** means a number came from a recorded run against real collected data, not an estimate. **Proposed** means it remains future work. The distinction matters because earlier planning documents describe sizes, counts and completion states the system has moved past, and because two of the three models fall short of validated.

Where a result is negative, it is reported as a result. The evaluation in Chapter 6 shows that one model generalises and two do not, and identifies the cause. That finding is presented directly rather than being obscured, because a report whose claims can be checked is more defensible than one whose claims cannot.

Acknowledgments

We thank our supervisor for guidance throughout the project, and the faculty of the Department of Software Engineering at Sir Syed University of Engineering & Technology for providing the academic framework in which this work was completed. We acknowledge the authors and maintainers of the open-source software on which the implementation depends, including PostgreSQL, the Rust ecosystem, PyTorch, ONNX Runtime and scikit-learn.

We also thank the participants who ran the tracking software on their personal machines for the duration of the collection period. Their consent made the dataset in this report possible, and the responsible-use constraints described in Section 3.10 exist because of the trust that participation implies.

Introduction to Group Members

Taqi Rahmani

2022F-BSE-283

Machine learning & preprocessing

Ibad Tahir

2022F-BSE-344

Data engineering & aggregation

Arqum Faisal

2022f-BSE-254

API, deployment & dashboard

Yagome Hina

2022F-BSE-305 Endpoint capture services

Certificate

SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY

University Road, Karachi-75300, Pakistan

Tel. : 4988000-2, 4982393-474583, Fax: (92-21)-4982393

Certificate of Completion

This is to certify that the following students

Taqi Rahmani 2022F-BSE-283

Ibad Tahir 2022F-BSE-344

Arqum Faisal 2022F-BSE-254

Yagome Hina 2022F-BSE-305

Have successfully completed the requirements for the Final Year Project titled

Intelligent Employee Productivity Tracking System (PTA)

In the report submitted for the Degree of Bachelor of Science in Software Engineering

---

Engr. Zainab Zakir

Sr. Lecturer

Department of Software Engineering, SSUET

Abstract

Conventional workforce monitoring records how long an employee is present, not how they work. It cannot separate focused work from idle presence, cannot establish whether input came from a person or an automation tool, and says nothing about developing capability. A systematic review of forty papers from 2020 to 2026 identified three resulting gaps: monitoring infers statistical outliers rather than intent, predictive HR analytics is dominated by attrition rather than growth, and endpoint behaviour is largely invisible to network-level security.

This project develops **PTA**, addressing those gaps with a tri-model architecture. Five Rust endpoint services capture process, keyboard, mouse, focus and derived-activity telemetry from consenting participants' Windows machines. Events reach a central PostgreSQL 16 database, where watermark-driven server-side functions aggregate them into three feature tables: fivesecond windows for automation detection, one-minute for productivity, daily rows for growth. Three models consume them — a TranAD reconstruction detector, a Temporal Fusion Transformer scorer, and a dilated-TCN forecaster paired with a transparent rule-based growth engine.

Evaluation uses a per-user chronological split over a 17-day, four-subject collection, comparing every model against an explicit baseline on all three splits. Cheat detection generalises: ROC-AUC **0.9558** on test, a train-to-test gap of 0.0008, PR-AUC 0.6377 against a 0.0988 random baseline, consistent across three independent automation vectors. The productivity model beats persistence on all three splits, though only narrowly; the growth model beats its baseline only on data it was fitted to — a memorisation signature attributable to dataset size — so growth ships as a rule-based engine whose every output traces to a named signal. Nine defects found by measurement are documented; two moved detection from chance to 0.9558. Three API instances and three ONNX models run on the project server behind a public dashboard.

Keywords: behavioural analytics, insider threat detection, anomaly detection, employee productivity, feature engineering, TranAD, temporal fusion transformer, explainable AI, ONNX

Contents

List of Figures

All nine figures are published at full resolution, in both vector and raster form, in the project repository: github.com/MuhammadTaqiRahmani/…/report-figures. Figures 1 and 8 are tall diagrams that cannot be reproduced at readable size within a page, so the report links to them there instead of embedding them.

List of Tables

11. Model generalisation across splits.
12. Model performance against baselines.
13. Cheat-detection performance per injected vector.
14. Recorded runtime measurements.
15. Lean validation plan.
16. Risks and mitigations.
17. Software requirements summary.

Chapter 1 Introduction

1.1 Overview

The conventional workplace was based on observation was taken away by transition to distributed and hybrid work. Two types of workforce management tools have emerged that have resulted in rest. opposing failures. Some workers enter the ”productivity theatre”, Creating something that conveys activity, but not work.. Monitoring tools have answered by becoming more invasive, capturing screenshots and raw keyboard input, which produces distrust and measurable backlash.

Measurement has evolved in response, from counting input, through output metrics drawn from workflow tools, to hybrid models combining behavioural logs with physiological and environmental signals. Section 2.2 examines that progression and its limits.

Two further considerations shape any credible design. Monitoring is adversarial: mouse jigglers, automation scripts and generative tools simulate work, and velocity-threshold detection reportedly falls below sixty per cent against them. And monitoring has a psychological cost — where autonomy feels threatened, the reactance literature reports counter-productive behaviour — so transparency is an engineering requirement, not a presentation choice.

1.2 Problem Statement

Existing approaches to workforce productivity analytics exhibit one or more of the following limitations:

- They measure presence and input volume rather than the character of the work performed;
- They cannot distinguish human-generated activity from automation, because the timing resolution required to detect machine regularity is absent from the stored data;
- They detect statistical outliers without inferring intent, producing false-alarm rates that make the output operationally unusable;
- Their predictive models are oriented toward negative outcomes such as attrition and underperformance, and offer little to an employee seeking to develop;
- They present model confidence as if it were a direct measurement of a person's capability or state;
- They provide no inspectable evidence for a score, so neither a manager nor the employee can audit a conclusion.

This project addresses those limitations by building an auditable pipeline that captures behaviour at a resolution sufficient to detect automation, aggregates it into engineered features keyed to a stable identity, and produces predictions accompanied by the evidence that supports them.

1.3 Objectives

The aim is to design, implement and evaluate an end-to-end system that captures endpoint behavioural telemetry from consenting participants, engineers it into multi-granularity feature representations, and applies machine learning to three distinct questions — current productivity, activity authenticity, and capability development — while reporting honestly on which of those questions the collected evidence can and cannot answer.

1.3.1 Objectives and Status **Table 1.** Project objectives and completion state.

Objective	Current status
Capture process, keyboard, mouse, focus and activity telemetry from Windows endpoints	Implemented and deployed on four machines
Store telemetry centrally with pooled, resilient connectivity	Implemented and deployed (PostgreSQL 16 with PgBouncer)
Engineer raw events into multi-granularity feature tables	Implemented and deployed (three tables, watermark-incremental)
Resolve a stable per-person identity across machines	Implemented, including collision handling and manual override
Build a cheat / automation detection model	Implemented, evaluated and deployed; generalises (Section 6.3)
Build a productivity scoring model	Implemented and deployed; does not beat its baseline on unseen data
Build a growth prediction and recommendation model	Neural model implemented but not validated; rule-based engine is the production path
Serve model output through an API	Implemented and deployed (three loadbalanced instances)
Provide a dashboard presenting results	Implemented and deployed at pta.emotionflow.site
Deploy trained models to the server	Implemented (ONNX, 152 MB runtime), execution verified on the server
Operate collection without manual disk supervision	Implemented (watermark-gated automated disk guard with off-site archival)
Establish labelled productivity accuracy against human annotation	Not completed — no human productivity labels exist
Establish growth-forecasting skill	Not completed — insufficient longitudinal data (Section 6.4)
Validate cheat detection against real-world evasion attempts	Not completed — evaluation uses synthetic injection

1.3.2 Research Questions The report is organised around five practical questions:

1. How can raw endpoint telemetry be aggregated into feature representations at the different time granularities that productivity, automation detection and growth each require?
2. How can a stable analytical identity be maintained when operating-system usernames are neither unique across machines nor stable over time?
3. Can automation-generated activity be distinguished from human activity using behavioural timing features alone, without content capture?
4. Can a productivity or growth signal be learned from a short collection window, and if not, what does the evidence say is required?
5. How should a system present a behavioural score so that the conclusion is auditable rather than asserted?

The evaluation answers questions one, two, three and five. It answers question four partially — productivity narrowly beats its baseline while growth does not — and quantifies the shortfall, which is itself the finding reported in Section 6.4.

1.4 System Features

The practical contribution is an integrated, measured system rather than a new model architecture. Its main contributions are:

1. A working endpoint-to-prediction pipeline in which every stage is reproducible, from raw capture through server-side aggregation to stored, versioned predictions;
2. A multi-granularity feature-engineering layer implemented as watermark-incremental serverside functions, so that aggregation is restartable and never reprocesses history;
3. A device-qualified canonical identity layer that resolves a real collision in which two distinct people shared an operating-system username;
4. An evaluation design in which every model is compared to an explicit baseline on all three splits, which is what allows two of the three models to be identified as memorising;
5. A synthetic automation-injection protocol requiring volume and regularity jointly over contiguous episodes, developed after the naive protocol was shown to be ill-posed;

6. A transparent rule-based growth engine in which each verdict carries the signal name, recent value, baseline value and effect size that produced it;
7. An evidence-first account of nine defects found by measurement, including the two that moved cheat detection from chance to 0.9558.

1.5 Project Scope

1.5.1 Included

- Endpoint telemetry capture on Windows for process, keyboard, mouse, window focus and derived activities;
- Central storage and server-side feature engineering into three granularities: five-second, oneminute and daily;
- Canonical device-qualified identity resolution with manual override;
- Three model architectures with training, evaluation against explicit baselines, and versioned prediction storage;
- A rule-based growth and recommendation engine with per-signal evidence;
- A read-only serving API, a results dashboard, and ONNX model deployment on the server;
- Automated disk management with watermark-gated pruning and off-site archival.

1.5.2 Excluded or Not Yet Validated

- Employment, disciplinary, hiring, appraisal or any other high-impact decision-making;
- Content capture: no screenshots, no keystroke content, no file contents, no network payloads;
- Labelled productivity accuracy against human annotation;
- Growth forecasting skill over a validated horizon;
- Real-world cheat prevalence or detection against a live adversary;
- Physiological and environmental sensing, which the literature associates with the highest Reported productivity-model accuracy;
- Privacy-preserving edge processing, local sensitive-application filtering, and personallyidentifiable-information anonymisation;
- Cross-platform support: macOS and Linux endpoints are not implemented;
- Production-scale availability, throughput and multi-tenant isolation.

1.6 Chapter Summary

The literature review is provided in Chapter 2 and the project gaps identified. Chapter 3 provides the principles and concepts of design, architecture, data and aggregation model, identity resolution, the Security and responsible-use limitations, model designs. Chapter 4 describes the Implemented services, pipelines and interfaces. Chapter 5 has been split into functional verification and Evaluates models and documents faults detected. Chapter 6 includes measured performance data. Defines the accuracy and validity limit. Chapter 7 frames commercialisation as unvalidated hypotheses. Chapter 8 defines future work in priority order and Chapter 9 concludes.

Chapter 2 Literature Review

2.1 Review Methodology

The literature underpinning this project was surveyed as a systematic review conducted according to PRISMA guidelines [1], [2]. Searches covered IEEE Xplore, MDPI and Elsevier ScienceDirect, using Boolean term groups across three areas: employee monitoring and productivity tracking with machine learning; insider-threat detection and behavioural biometrics with mouse and keystroke dynamics; and employee attrition or performance prediction with deep learning and explainable AI. Cross-referencing was used to identify preprints and grey literature reflecting trends not yet indexed.

Inclusion was restricted to full-text English work published between 2020 and 2026 that addressed automated monitoring, machine-learning prediction or behavioural security architectures, and that took the form of an empirical study, a systematic review or a validated architectural framework. Opinion pieces, non-peer-reviewed

commercial whitepapers, pre-2020 physical-surveillance work and purely managerial performance papers without computational content were excluded. A threelevel quality assessment process eliminated 22 candidate papers of 62 submitted, and This leaves 40 papers to be synthesized. Extraction recorded methodology, dataset and Identified research gap for each paper

2.2 The Evolution of Productivity Metrics

The work reviewed is logically organized. First-generation systems (roughly 2010–2020) Utilized input proxies: keystrokes per minute, number of clicks, window activation time, and presence indicators. These are described in literature as high noise and low value, easily vanquished by, Not very complex automation, structurally cannot represent deep work [2]. Second-generation systems from 2021 moved to output metrics via workflow-tool integration and task mining, improving validity but lagging real time and inviting quota-driven behaviour such as committing low-quality code to satisfy a measure [38].

The current state of the art combines behavioural logs with physiological and environmental signals. One study integrating heart-rate variability from wearables with behavioural logs reports productivity prediction improving from 81 per cent to 96 per cent, suggesting that stress and focus are better predictors of productivity than activity volume [3]. Work on environmental context argues that digital-only models cannot account for physical blockers and therefore misattribute environmental disruption to disengagement [6].

> **Scope note.** This project operates at the second generation of data with third-generation modelling techniques. It has no physiological or environmental sensing, and the accuracy gains that the literature associates with those modalities are therefore unavailable to it. This is relevant when interpreting the productivity result in Section 6.3 and is stated again there.

2.3 Data Collection Architectures

Three architectural patterns appear in the literature, trading visibility against privacy and cost. Network sniffing is minimally intrusive but increasingly blinded by encryption and VPNs, and cannot see offline activity [16]. Browser extensions deploy easily but see only web contexts, missing an estimated forty per cent of desktop workflow activity [26]. Kernel-level agents give full visibility into file systems, peripherals and application internals at under two per cent CPU for optimised Rust or C, cutting time-to-detect from minutes to milliseconds against cloud polling [37].

The literature also emphasises that a tracker resides on the user's device, which is hostile territory in security terms, and therefore requires anti-tamper mechanisms and cryptographic log integrity [10], [35]. Separately, privacy-by-design guidance from the EU Joint Research Centre recommends local filtering of sensitive applications before data enters memory, anonymisation of personally identifiable information, and edge processing that transmits only derived metadata, reported to reduce transmission volume by approximately ninety per cent [26], [32].

2.4 Insider Threats and Cheat Detection

Remote work has produced an arms race between monitoring tools and evasion techniques. Hardware mouse jigglers, VPN masking and automation scripts simulate presence, and traditional mouse logging cannot detect a mechanical device that moves the cursor while keeping the USB connection active. Heuristic detection based on velocity thresholds is reported below sixty per cent detection rate against such devices [2].

Learned approaches perform substantially better. Work using generative adversarial networks to synthesise bot trajectories and train a discriminator reports area under the curve above 0.95, and identifies the discriminating property directly: human activity carries entropic noise arising from natural irregularity, whereas mechanical automation exhibits superhuman regularity [22]. Continuous authentication using keystroke and mouse dynamics has been shown to identify an impostor within twenty to sixty seconds of account takeover [4], [13].

A recurring limitation is false-alarm rate. Anomalous behaviour, such as working at an unusual hour or accessing large files, is not necessarily a threat. Work on the Deltrux framework and on deep evidential clustering argues for representing uncertainty explicitly, so that a system can separate a clumsy user with a high anomaly score and low malice probability from a malicious insider with both [12], [16].

2.5 Predictive Analytics for Employee Growth

Predictive HR analytics is dominated by attrition modelling, typically as binary classification with static models such as random forests, reported at around 91 per cent accuracy [20]. Work on promotion prediction argues that this deficit orientation cannot explain positive contribution [23]. Newer approaches use LSTM and Temporal Fusion Transformer architectures over time-series performance data to detect burnout trajectories weeks

in advance and to model skill-acquisition curves, with reported accuracy in the 94 to 97 per cent range over a four-to-six-week prediction window [24].

Two further findings shape design. Integrating satisfaction measurement directly into the prediction pipeline is reported to increase prediction stability by fifteen per cent [28]. And explainability is treated as a precondition for adoption rather than an enhancement: promotion and intervention decisions require a defensible rationale, and applying SHAP values to employee prediction models is presented as the mechanism for converting an opaque judgement into concrete, actionable feedback [5].

2.6 Ethical Implications and Worker Resistance

The reviewed literature considers the psychological cost of monitoring as the major failure mode rather than a secondary issue. Psychological reactance theory predicts that employees who When they feel their independence is being challenged, will engage in counter-productive work behaviours to lose a sense of control, which leads to a loss of morale, increased stress and poor quality of output. Despite an increase in activity measured [11,34]. The observer effect adds to this: monitoring the object affects the phenomenon. Changes the behaviour being measured, which compromises the validity of the measure.

Qualitative research indicates that employees are not passive objects of, but rather active participants in the analysis process. Surveillance, reporting on performative compliance to technical evasion [35]. Survey evidence indicates that transparency is the predominant element of acceptance: systems that explain why a score changed are received as supporting tools, systems with opaque interfaces, with no explanation, are not. Feedback are received as a spyware [27]. Adhering to labour and privacy law is shown as It is a necessary, but not sufficient, condition [32, 36].

2.7 Research Gaps and Positioning

Three gaps emerge from the synthesis, and they define this project's targets:

Table 3. Research gaps and implementation response.

Research gap	Description	Implementation response in this project
The intent-detection gap	Anomaly detectors identify statistical outliers but cannot infer intent, generating high false-alarm rates that overwhelm reviewers [30].	An unsupervised reconstruction model trained on normal behaviour, evaluated against three independently constructed automation vectors, and reported as a triage signal with a stated false-positive rate rather than a verdict (Sections 3.8 and 6.3).
The deficit model of HR analytics	Predictive models target attrition, fraud and under-performance; models identifying development or promotion readiness are scarce [23], [25].	A growth model and a rule-based growth engine oriented to capability development, with engagement explicitly weighted at zero so that the score cannot reward overwork (Sections 3.9 and 4.7).
The endpointnetwork disconnect	Zero-trust architectures emphasise network access control, but network gates cannot see offline endpoint behaviour [16], [26].	Kernel-adjacent Rust endpoint agents providing full-system visibility including offline periods, with telemetry synchronised centrally (Sections 3.3 and 4.2).

The review also suggests a tri-model architecture for a next generation system: a hybrid scoring model were evaluated.Unsupervised security and cheat-detection model, a temporal productivity scoring model were evaluated. growth-prediction model with fairness metrics and userexplainable. This project implements actually have.Chapter 6 reports on which of the three the project implements actually have, and which ones implement the tri-model structure. collected evidence supports.

2.8 Literature Synthesis

Table 4. Literature synthesis.

Area	Lesson from prior work	PTA design response	Remaining limitation
Metric generation	Input proxies are gameable; hybrid models perform best	Behavioural feature engineering at three granularities	No physiological or environmental modality
Collection architecture	Endpoint agents give full visibility at low overhead	Five Rust endpoint services with a supervising launcher	Windows only; services crash during 17–44% of active hours
Privacy by design	Local filtering, anonymisation and edge processing recommended	No content capture: timing and metadata only	No local sensitive-app filter, anonymisation or edge processing
Cheat detection	Regularity distinguishes automation; heuristics fail	Unsupervised reconstruction over 5-second timing features	Synthetic injection, not a live adversary
Uncertainty	Anomaly must be separated from malice	Continuous score with reported precision; triage not verdict	No explicit maliceprobability model
Growth modelling	Temporal models capture skill-acquisition curves	Dilated-TCN forecaster plus rule-based engine	24 training days; neural model not validated
Explainability	Transparency governs acceptance	Per-signal evidence on every growth verdict	No SHAP attribution for the neural models
Worker acceptance	Reactance and the observer effect degrade validity	Consent-based participation; no content capture	No employee-facing dashboard or fairness audit

Chapter 3 Design

3.1 Design Methodology and Software Process Model

There are six principles behind the design of the system:

- **Metadata, not content.** The system records timing, counts and window metadata. It never captures keystroke content, screenshots, file contents or network payloads. This bounds privacy exposure at the point of collection rather than downstream.
- **Aggregate where the data is.** Feature engineering runs as server-side database functions rather than in an application layer, so that no raw event ever crosses the network twice and aggregation cannot drift from storage.
- **Incremental and restartable.** Every pipeline is driven by a watermark, so a crashed or interrupted run resumes without reprocessing history and without gaps.
- **Identity is an input, not an output.** Aggregation is scoped per device before any grouping occurs. Assigning identity after aggregation was the defect described in Section 5.5 and is structurally excluded by the current design.
- **Evidence accompanies every score.** A prediction that cannot be traced to the signals that produced it cannot be audited, and the literature identifies opacity as the dominant cause of rejection.
- **Fail closed on destructive operations.** Automated pruning requires positive proof that data has been consumed by every downstream pipeline, and does nothing in the absence of that proof.

3.2 Architectural Design / Design Patterns

The system is a four-stage pipeline: capture, store, aggregate, and predict and serve. Capture runs on participants' Windows machines; the remaining three stages run on a central server.

Figure 1 is a tall diagram and is published online at full resolution rather than reproduced here.

View figure (vector, zoomable) Download PNG

Figure 1. PTA end-to-end architecture. Endpoint services capture behavioural telemetry, which is stored centrally, aggregated by server-side functions into three engineered feature tables, consumed by three models, and served through a read-only API to the dashboard.

3.3 Process Flow / Representation

Five independent services run on each monitored machine under a supervising launcher. Independence is deliberate: a fault in one capture domain does not stop the others, and each service can be updated separately.

Service	Captures	Design rationale
Process monitoring	Process lifecycle, hierarchy, resource usage via ETW	Application context is required to interpret any input signal
Keyboard input	Key event timing, dwell and flight intervals — no key content	Inter-key interval regularity is the primary automation signal
Mouse input	Movement kinematics, click timing and accuracy	Trajectory smoothness distinguishes human from synthetic motion
Window focus	Focus transitions, dwell duration, application identity	Focus duration is the basis of deepwork and attention features
Activities	Derived activity records joining process and focus context	Provides a higher-level semantic layer over raw events
Launcher	Supervises the five services	Single point of control for start, restart and configuration

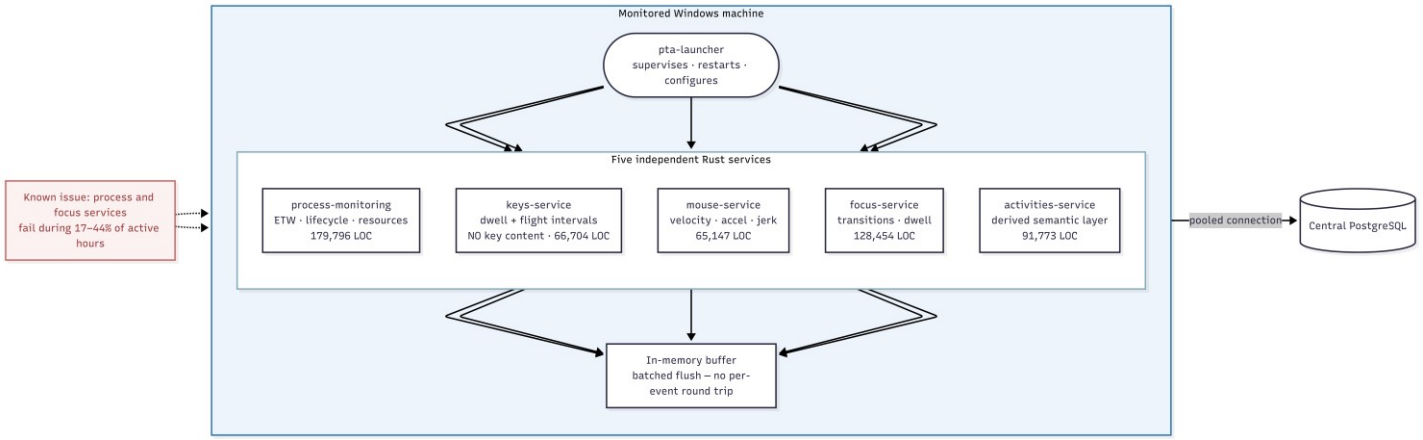


Figure 2. Endpoint tracker topology. Five independent capture services report to a supervising launcher and write through a pooled connection to central storage.

The literature recommends anti-tamper mechanisms and cryptographic signing of logs before they leave the endpoint [10], [35]. Neither is implemented here; the deployment is a consenting research cohort rather than an adversarial workforce, and this is recorded as a limitation in Section 3.11 and as future work in Chapter 8.

3.3.1 Storage and Connection Design All telemetry is written to a central PostgreSQL 16 database. Connecting to endpoint services: PgBouncer used in transaction pooling mode (many short-lived connections from) Multiple machines and without using up connection slots on the server.

By default, transaction pooling does not hold session state because this is traditionally considered to be a problem. statements. Instead of disabling prepared statements (a common workaround) which would force Uses named prepared statements, but re-plans the query on each execution. Keeping the pooler's prepared-statement support and retaining pooling efficiency and plan caching. One related limitation is actually directly written into the code: the pooler doesn't accept statement timeout as a startup parameter, so timeout is imposed when connection is recycles or TCP keepalive is used. instead.

3.4 Design Models [along with description]

3.4.1 Productivity Model Design The productivity model is a Temporal Fusion Transformer-style architecture combining a variableselection network, a bidirectional LSTM encoder and temporal attention, operating over sequences of one-minute feature windows.

Its central difficulty is the target. No human productivity annotations exist, and the obvious alternative — deriving a label from the same features the model receives as input — is circular, scoring well while demonstrating nothing. The design uses a forward-looking proxy: mean input activity over the five following windows, computed only from future windows and never present in the feature set. That makes the task honest forecasting, and makes persistence the baseline to beat.

3.4.2 Cheat Detection Model Design Cheat detection is designed as unsupervised anomaly detection because labelled cheating does not exist and because a supervised classifier could only recognise evasion strategies present in training. The model is TranAD, a transformer architecture that learns to reconstruct normal behavioural sequences; reconstruction error becomes the anomaly score.

Evaluation requires known anomalies, produced by synthetic injection, and that injection design is itself a substantive result. The naive approach — forcing perfect regularity by driving inter-key interval variation to zero — proved ill-posed: twenty-two per cent of genuinely active windows already show zero interval variation for innocent reasons, such as containing a single keystroke. The corrected protocol requires volume and regularity together over contiguous two-minute episodes, across three independently constructed vectors, so performance cannot be attributed to one artefact.

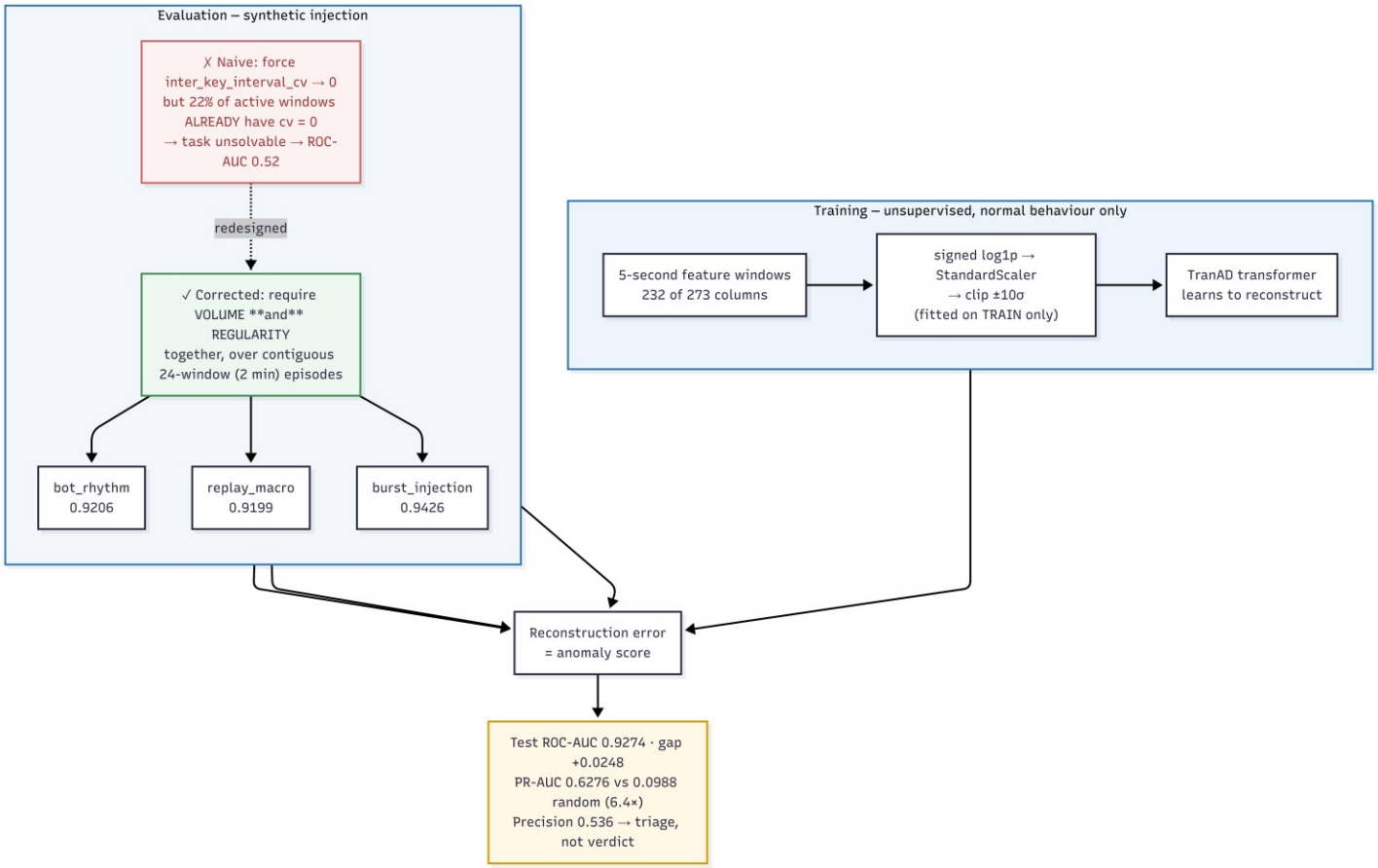


Figure 5. Cheat-detection design: unsupervised reconstruction training on normal behaviour, with three synthetic automation vectors injected as episodes for evaluation.

3.4.3 Growth Model Design Growth is designed with two engines serving the same interface. The neural engine is a dilated temporal convolutional network combined with an N-HiTS forecasting head, intended to model skill-acquisition curves over a thirty-day context as the literature describes [24]. The rule-based engine compares each subject only to their own earlier behaviour using a robust effect size — the shift in median divided by the interquartile spread of the baseline period — aggregated across four weighted dimensions: skill acquisition, focus quality, consistency and work quality.

Two design decisions are worth stating. First, engagement — tracked minutes and longest continuous active period — is computed and reported but weighted at zero, because more hours is effort rather than improvement and weighting it would reward overwork. Second, signal selection is evidence-driven: a first version built on the intuitively obvious columns returned an identical neutral score for every subject, because those columns were never populated by the aggregation layer. The engine was rebuilt on columns verified as populated and varying.

Both engines write to the same table under different version identifiers, so they can be queried side by side and compared directly. Chapter 6 explains why the rule engine is the production path.

3.5 Data Design

The three analytical questions require different time resolutions, and this drives the central design decision of the data layer. Automation detection depends on sub-second timing regularity and is destroyed by coarse aggregation. Productivity is meaningless at five-second resolution and natural at one minute. Growth is a multi-week quantity for which a daily row is the appropriate unit. The system therefore maintains three feature tables rather than one.

Table 2. Engineered feature table inventory.

Feature table	Window	Declared columns	Consumer
cheat_detection_features	5 seconds	321	Cheat Detection Model (CDM)
unified_time_series	1 minute	211	Productivity Scoring Model (PSM)
employee_growth_features	1 day	214	Growth Prediction and Recommendation (GPRM)

Each table is populated by a server-side PL/pgSQL function driven by a watermark recording the last successfully processed window. A run processes only the interval between the watermark and the present, then advances it, so that aggregation is incremental, restartable and idempotent. Aggregation iterates per device and stages results before insertion, guaranteeing that no output row can blend behaviour from more than one machine.

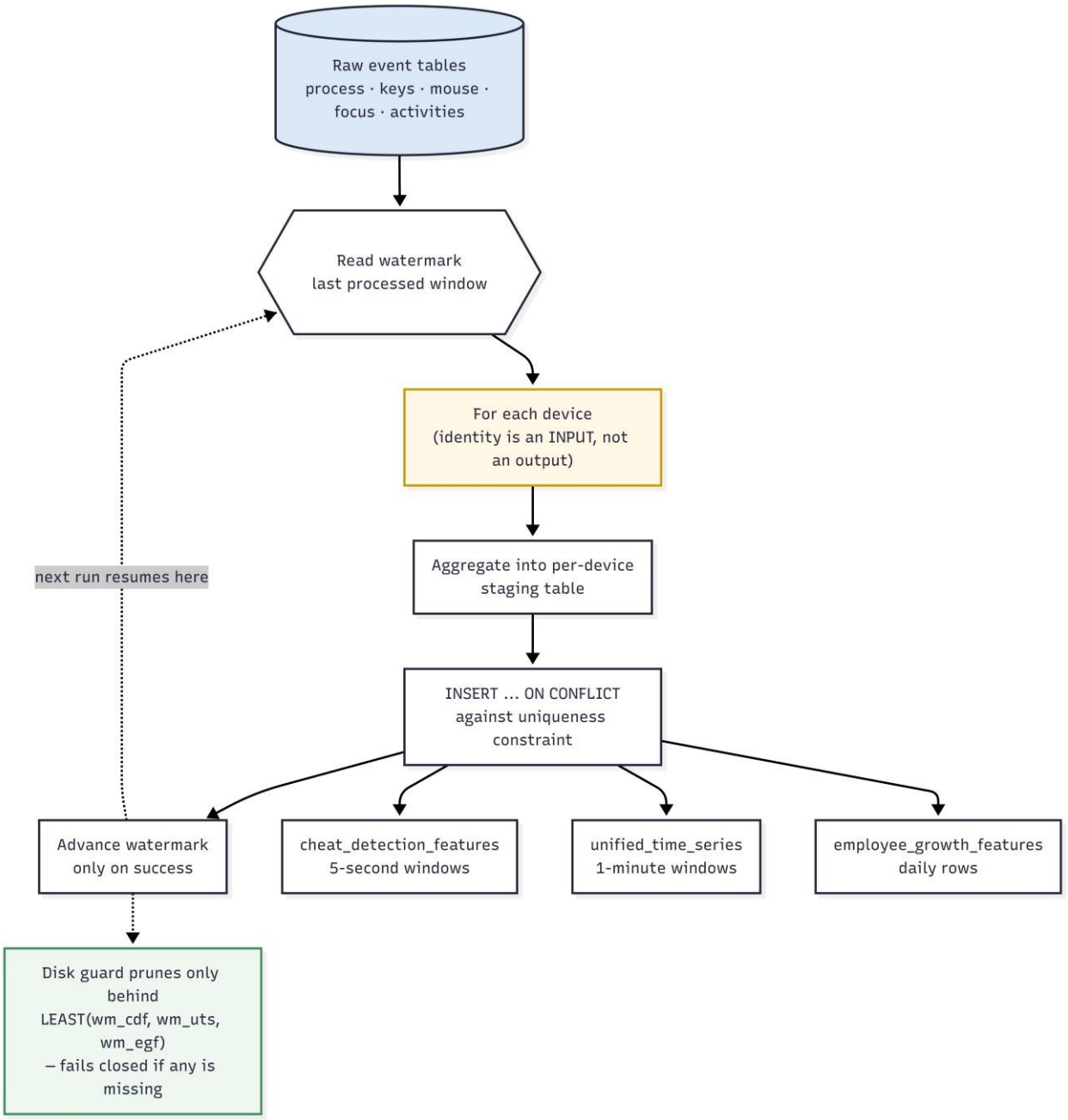


Figure 3. Watermark-driven aggregation. Raw event tables are aggregated per device into three engineered feature tables at three granularities, each advancing its own watermark.

3.5.1 Identity Design Analytical identity cannot be taken directly from the operating-system username. Usernames are not unique across machines, and during this project two distinct participants were found to have configured the same Windows account name, causing both to resolve to a single analytical subject. Merging two people's behavioural profiles would not only corrupt each profile but would make ordinary variation in one appear as an anomaly in the other.

Identity resolution will then identify colliding usernames and qualify with the device that created them. only where there is a collision. The qualification is deliberately restrictive: applying it unconditionally would make one person seem like a different person every time he or she changed. Machine, swap one with another identity error. The identity table is the table that holds the identity of which rule two subjects that collided are device-qualified; produced each mapping, the two colliding subjects are marked as device-qualified; produced each mapping, the two colliding subjects are marked as device-qualified. The remaining two are settled directly, the difference between the two is explicit.

In the case of a rule that can't be determined automatically, A manual override table is employed. Canonical views expose original values in raw tables are retained while the resolved identity are supplied to consumers. is auditable and reversible — important property, as an identity decision can't be reversed. If used in conjunction

with re-visited, all downstream results would be unfalsifiable.

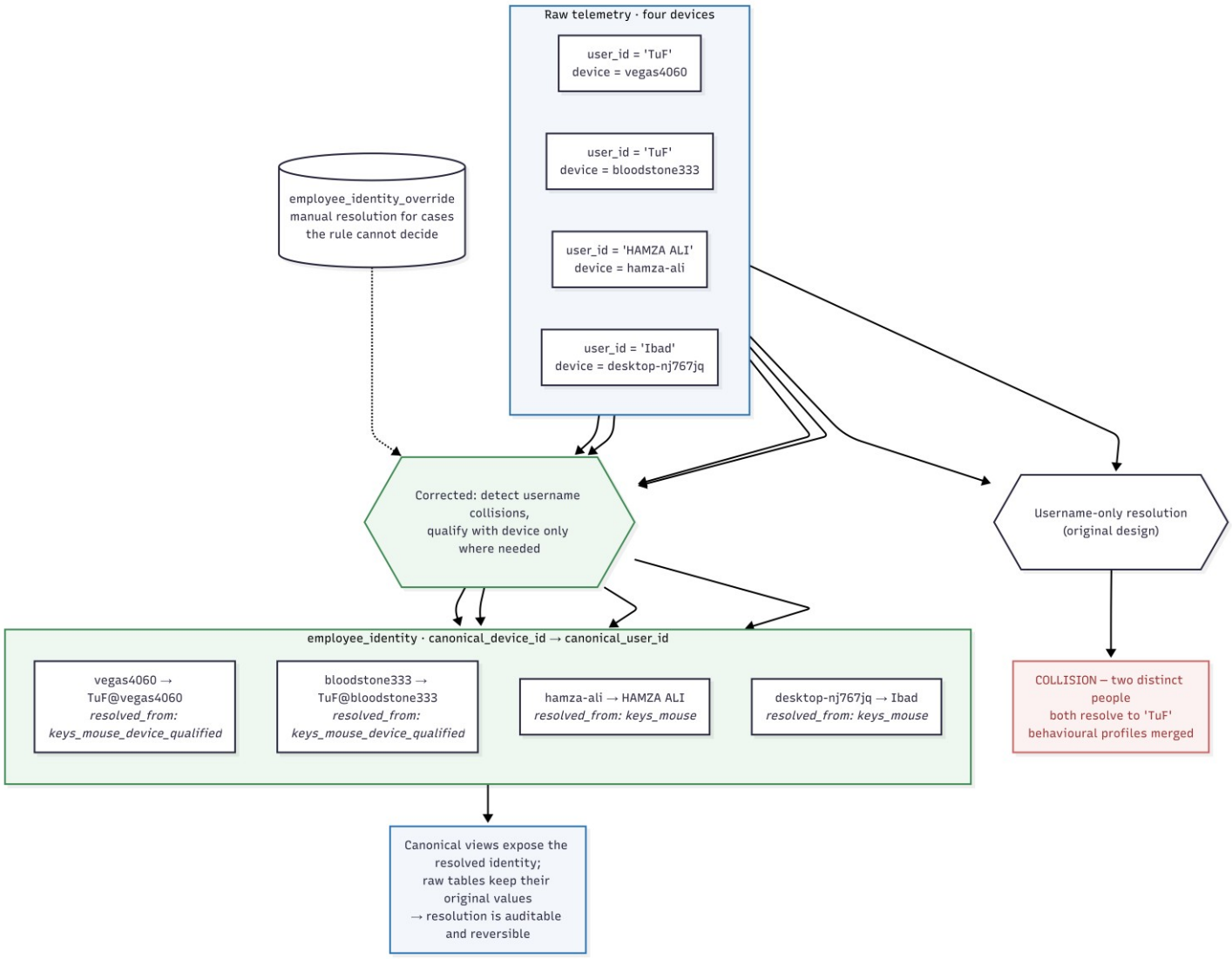


Figure 4. Canonical device-centric identity resolution, including the collision case in which two distinct people share an operating-system username.

3.6 Security, Privacy and Responsible Use

3.6.1 Implemented Controls

- no content capture at any layer: no keystroke content, screenshots, file contents or network payloads;
- pooled database access with credentials held server-side, not embedded in endpoint binaries distributed to participants;
- a read-only serving API: the public interface rejects all non-idempotent HTTP methods at the reverse proxy;
- vhost isolation on a shared server, including a catch-all TLS listener that refuses requests for any domain other than this project's;
- consent-based participation with participants aware of what is captured.

3.6.2 Known Gaps

- no anti-tamper protection or cryptographic log signing at the endpoint;
- no local filtering of sensitive applications before capture, as privacy-by-design guidance recommends [32];
- no anonymisation or pseudonymisation of identity in the analytical layer;
- no edge processing: raw events are transmitted centrally rather than reduced at source;

- a self-signed origin certificate, which is correct for the current proxy configuration but not for strict origin validation;
- no data-retention policy, deletion workflow or access logging.

  Responsible use. PTA produces model estimates of behavioural patterns. It must not be used as the sole or primary basis for employment, disciplinary, appraisal, hiring or security decisions. The cheat-detection score is a triage signal with a measured false-positive rate of approximately forty-six per cent at the operating threshold, and is not evidence of misconduct. The growth assessment is a transparent summary of measured behavioural change, not a measurement of a person's capability or potential. Any consequential use requires human review, informed consent and independent validation.

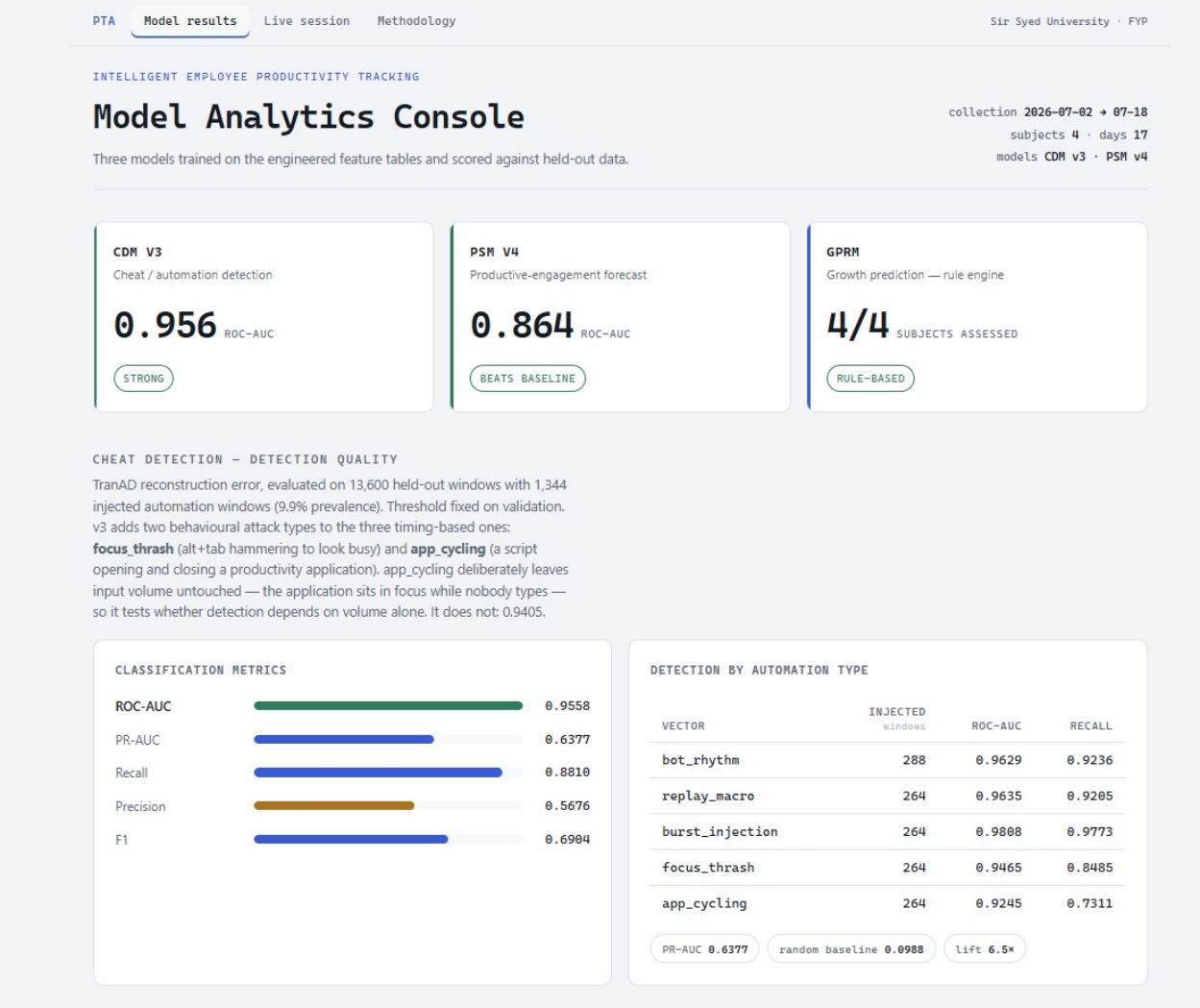
3.7 Interface Design

The serving interface is a read-only HTTP API exposing four endpoints per subject — cheat scores, productivity scores, growth assessment and alerts — each accepting a model-version parameter and a time window. Inference is deliberately separated from serving: models write to prediction tables and the API only reads them, so serving latency is bounded and independent of model execution, and a model can be re-run, versioned or rolled back without touching the serving path.

The dashboard presents per-model results with train, validation and test metrics shown together rather than reporting only the best figure, because the generalisation gap is the actual finding for two of the three models. A companion methodology page documents the splits, preprocessing, injection design and evaluation protocol.

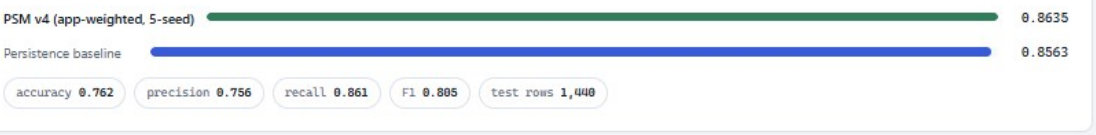
The dashboard is publicly accessible at pta.emotionflow.site, where the Chapter 6 results can be inspected against the stored predictions.

S



PRODUCTIVITY — ENGAGEMENT FORECAST VS BASELINES

Task: will this person show more *productive* engagement over the next five minutes than their own recent usual? Input is weighted by application before forecasting — development counts fully, browser and system half, gaming and social zero — so a gaming burst no longer reads as productivity. v4 is a five-seed ensemble; on held-out data gaming's score fell from 0.566 to 0.314 while development rose to 0.438.



accuracy **0.762** | precision **0.756** | recall **0.861** | F1 **0.805** | test rows **1,440**

GROWTH — FORECAST ERROR VS BASELINES

Daily-granularity forecasting. Lower mean absolute error is better. Evaluated on 8 held-out days.

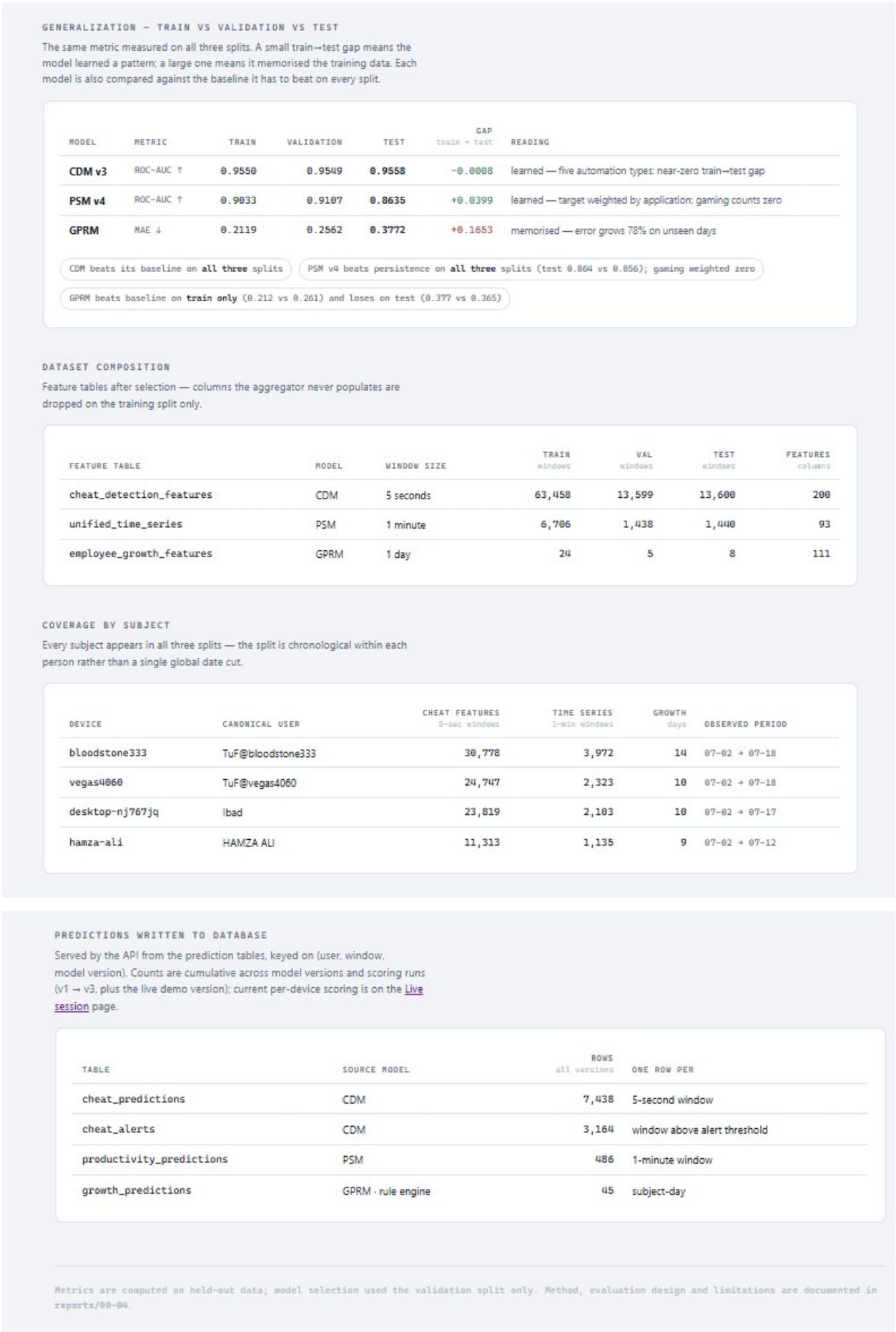
SCORED OUTPUT

Live per-device cheat and productivity scoring — refreshed automatically as trackers run — is on the Live session page, verified against the database on every load. Below is the production growth assessment, computed from each subject's accumulated daily history rather than a live window.

GROWTH ENGINE — RULE-BASED (PRODUCTION)

SUBJECT	DAYS observed	GROWTH 0–1	TRAJECTORY	LEADING EVIDENCE
Ibad	10	0.698	steady_growth	tracked minutes 15 → 110 (+3.0)
HAMZA ALI	9	0.462	stagnant	new tools 40 → 49, but longest focus 77,972 ms → 0
TuF@vegas4060	10	0.339	declining	burnout composite 0.62; tool adoption 1.30 → 1.17
TuF@bloodstone333	14	0.300	declining	start-time consistency 2.9 → 5.85 (−3.0)

within-subject baselines | robust effect size **median shift + baseline IQR** | engagement weighted **0** — hours are not growth



GENERALIZATION — TRAIN VS VALIDATION VS TEST

The same metric measured on all three splits. A small train→test gap means the model learned a pattern; a large one means it memorised the training data. Each model is also compared against the baseline it has to beat on every split.

MODEL	METRIC	TRAIN	VALIDATION	TEST	GAP train → test	READING
CDM v3	ROC-AUC ↑	0.9550	0.9549	0.9558	−0.0008	learned — five automation types; near-zero train→test gap
PSM v4	ROC-AUC ↑	0.9033	0.9107	0.8635	+0.0399	learned — target weighted by application; gaming counts zero
GPRM	MAE ↓	0.2119	0.2562	0.3772	+0.1653	memorised — error grows 78% on unseen days

CDM beats its baseline on **all three** splits

PSM v4 beats persistence on **all three** splits (test 0.864 vs 0.856); gaming weighted zero

GPRM beats baseline on **train only** (0.212 vs 0.261) and loses on test (0.377 vs 0.365)

DATASET COMPOSITION

Feature tables after selection — columns the aggregator never populates are dropped on the training split only.

FEATURE TABLE	MODEL	WINDOW SIZE	TRAIN windows	VAL windows	TEST windows	FEATURES columns
cheat_detection_features	CDM	5 seconds	63,458	13,599	13,600	200
unified_time_series	PSM	1 minute	6,706	1,438	1,440	93
employee_growth_features	GPRM	1 day	24	5	8	111

COVERAGE BY SUBJECT

Every subject appears in all three splits — the split is chronological within each person rather than a single global date cut.

DEVICE	CANONICAL USER	CHEAT FEATURES 5-sec windows	TIME SERIES 1-min windows	GROWTH days	OBSERVED PERIOD
bloodstone333	TuF@bloodstone333	30,778	3,972	14	07-02 → 07-18
vegas4060	TuF@vegas4060	24,747	2,323	10	07-02 → 07-18
desktop-nj767jq	Ibad	23,819	2,103	10	07-02 → 07-17
hamza-ali	HAMZA ALI	11,313	1,135	9	07-02 → 07-12

PREDICTIONS WRITTEN TO DATABASE

Served by the API from the prediction tables, keyed on (user, window, model version). Counts are cumulative across model versions and scoring runs (v1 → v3, plus the live demo version); current per-device scoring is on the Live session page.

TABLE	SOURCE MODEL	ROWS all versions	ONE ROW PER
cheat_predictions	CDM	7,438	5-second window
cheat_alerts	CDM	3,164	window above alert threshold
productivity_predictions	PSM	486	1-minute window
growth_predictions	GPRM · rule engine	45	subject-day

Metrics are computed on held-out data; model selection used the validation split only. Method, evaluation design and limitations are documented in reports/00–04.

Figure 6. Dashboard results page, showing per-split metrics for each model alongside its baseline, and the growth assessments produced by the rule-based engine.

Chapter 4 System Development

4.1 Technology Stack

Table 5. Technology stack.

Layer	Main technologies	Purpose
Endpoint capture	Rust 2024, Tokio, windows-rs, sqlx	Five monitoring services and the supervising launcher
Storage	PostgreSQL 16	Raw events, features, identity, watermarks, predictions
Pooling	PgBouncer (transaction mode)	Connection multiplexing from many endpoints
Feature engineering	PL/pgSQL	Server-side watermark-incremental aggregation
ML training	Python, PyTorch, scikit-learn, pandas	Model implementation, training and evaluation
Model serving	ONNX, onnxruntime	Portable graph execution on the server without a GPU
API	Rust, Axum	Read-only HTTP serving of stored predictions
Web	nginx, Cloudflare	TLS termination, static hosting, reverse proxy
Operations	systemd, cron, rclone	Service supervision, disk guard, off-site archival

4.2 Endpoint Services

The capture layer is implemented in Rust for two reasons drawn from the literature: endpoint agents must impose minimal overhead, reported at under two per cent CPU for optimised native implementations [37], and memory safety matters in a process that runs continuously on a participant's personal machine. Each service buffers events in memory and flushes in batches, so that per-event database round trips do not appear in the interaction path.

Table 6. Codebase metrics.

Component	Lines of code	Language
Process monitoring service	179,796	Rust
Focus monitoring service	128,454	Rust
Activities monitoring service	91,773	Rust
Keyboard input service	66,704	Rust
Mouse input service	65,147	Rust
Launcher and foundation	1,060	Rust
Total Rust	**532,934**	
Aggregation functions	4,087	PL/pgSQL
Schema definition and verification SQL	2,210	SQL
Grand total	**~539,231**	

4.3 Aggregation Pipelines

Three PL/pgSQL functions implement the feature-engineering layer. Each reads its watermark, iterates the interval between that watermark and the present, aggregates per device into a staging table, inserts with conflict handling against a uniqueness constraint, and advances the watermark only on success.

Table 7. Aggregation function inventory.

Function	Lines	Output
populate_cheat_detection_features()	2,259	5-second automation-detection features
populate_employee_growth_features()	1,288	Daily growth features
populate_unified_time_series()	540	1-minute productivity features

The design of these functions changed materially during the project. The original form aggregated globally and resolved identity afterwards, and joined component aggregates at a coarser grain than they were grouped at. Both proved to be defects that produced structurally valid but semantically wrong output; they are documented in Sections 5.4 and 5.5.

4.4 Machine Learning Pipeline

The modelling work is implemented as eight sequential, individually runnable phase scripts, so that any stage can be reproduced without re-running the others. The pipeline covers data foundation, label construction, the three model trainings, scoring, deployment verification and the rule-based growth engine.

4.4.1 Preprocessing Behavioural event counters are heavy-tailed by nature, and standard scaling on raw counts was found to collapse under that distribution, with one feature reaching 10,595 standard deviations on a validation window. Preprocessing therefore applies a signed logarithmic transform that compresses the tail while preserving sign and handling zeros, then standard scaling, then clipping at ten standard deviations. All parameters are fitted on the training split only and travel with the exported model.

4.4.2 Feature Selection Columns that are never populated or have zero variance are dropped, determined on the training split alone so that validation and test data cannot influence selection. Of 273 numeric columns available to the cheat model, 200 survive; of 193 available to the productivity model, 93 survive.

4.5 Model Deployment

Training and serving have opposing requirements: training needs a full deep-learning stack, serving must execute a fixed graph cheaply on a shared server with no GPU. All three models are exported to ONNX, cutting the server footprint to 152 MB against roughly 2 GB for a full framework. Each ships with its fitted preprocessing parameters and a manifest of version, training date, input shape and metrics — a graph without its transform is not reproducible. Deployment was verified by executing each graph on the server against a real batch.

4.6 Serving API and Dashboard

The API is implemented in Rust with Axum and runs as three systemd instances behind an nginx upstream. It reads prediction tables and performs no inference. The public dashboard presents model results and a methodology page, served from the same vhost with the API proxied readonly beneath it.

Both pages are live at pta.emotionflow.site. The API path beneath them rejects every nonidempotent HTTP method, so the public surface can read predictions but not alter them.

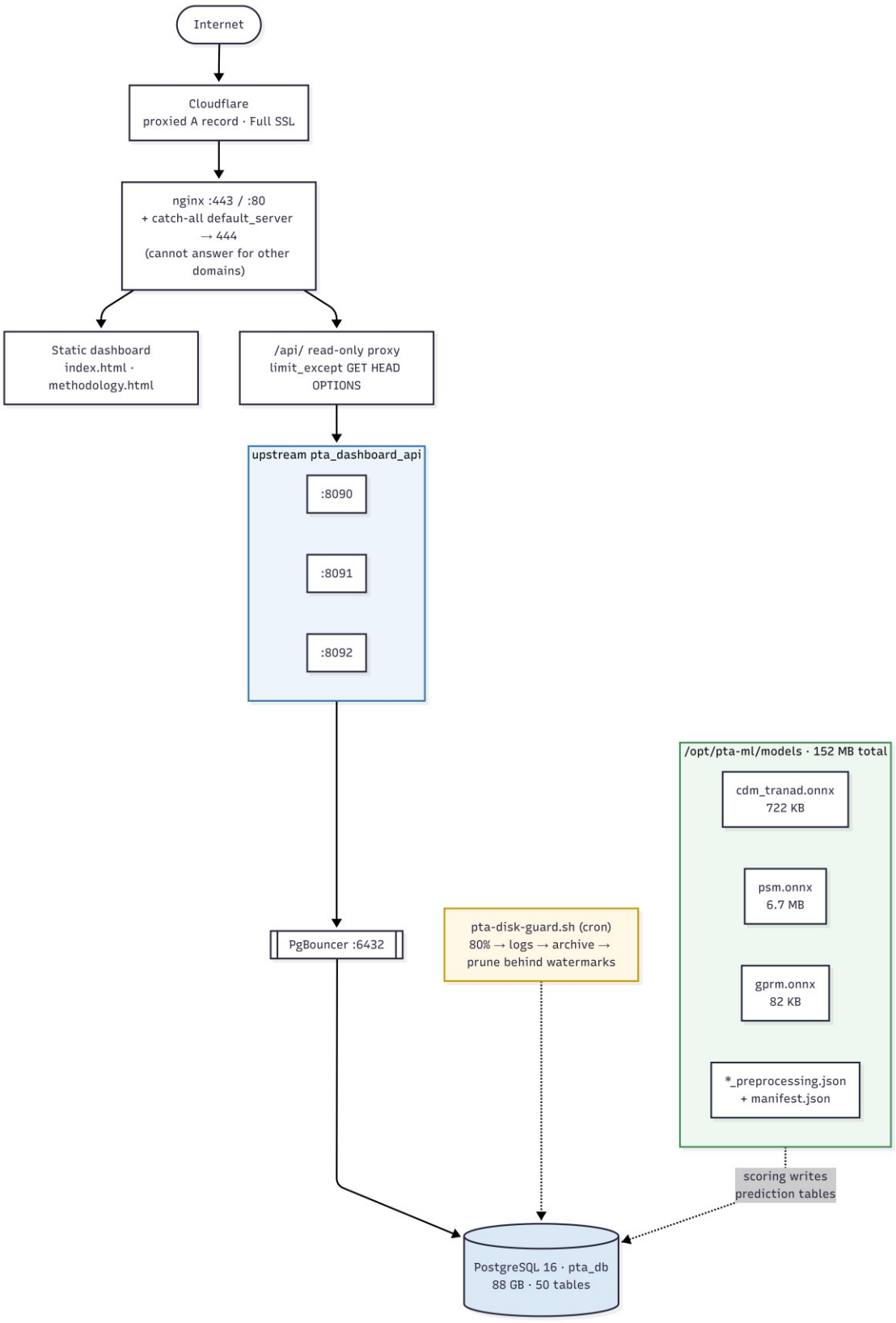


Figure 7. Server deployment topology: proxied DNS and TLS termination, static dashboard, read-only API proxy across three load-balanced instances, connection pooler and database.

4.7 Rule-Based Growth Engine

For each subject the engine computes a robust effect size per signal against that subject's own earlier baseline, orients every signal so positive means improvement, takes a median within each dimension, and blends the dimensions using fixed weights. Volatility gates the strong labels, so a large movement inside an erratic series reads as stagnant rather than accelerating — the principal way a rule engine misleads, guarded explicitly. Confidence

follows from observed days, and below four days the engine declines to score at all.

Every assessment carries the signals that produced it, each with its recent value, baseline value and effect size, and every recommendation stores its trigger condition and the observed value that fired it.

4.8 Development Responsibilities

Work stream	Primary owner	Integration dependency
Endpoint capture services and launcher		Database schema and connection policy
Database, aggregation and identity layer		Raw event schemas; feature contracts
Machine learning and evaluation		Feature table stability and split policy
API, deployment and dashboard		Prediction table schemas and model versioning
Integration, testing and documentation	Entire team	Frozen release candidate

Chapter 5 Testing

5.1 Manual Testing

Testing is separated into data-foundation verification, functional and integration checks, defect investigation, and model evaluation. The separation matters: a passing integration test says nothing about whether a model has learned anything, and conflating the two is a common way for reports to overstate their results. Model evaluation is reported separately in Chapter 6.

5.1.1 Unit Testing Before any model was trained, nine automated checks were run against every feature table. These target the failure modes that silently invalidate machine-learning results — leakage, missing subjects, incorrect scaling and shape errors — and all passed on live data.

Table 9. Data-foundation verification matrix.

#	Check	CDM	PSM	GPRM
1	Split is chronological within every user (no future leakage)	PASS	PASS	PASS
2	All four subjects present in train, validation and test	PASS	PASS	PASS
3	At least one informative feature survives selection	PASS	PASS	PASS
4	Dropped columns never reach the feature matrix	PASS	PASS	PASS
5	No zero-variance feature remains	PASS	PASS	PASS
6	Scaler centred on train (train mean approximately zero)	PASS	PASS	PASS
7	Identical column order across all splits	PASS	PASS	PASS
8	No missing values in any split	PASS	PASS	PASS
9	One sequence per row, correct shape, unique end indices	PASS	PASS	n/a

5.1.2 Integration Testing

Check	Result	Interpretation
Aggregation watermark advance	Passed	All three pipelines advance and resume correctly after interruption
Per-device row separation	Passed	9,533 rows across 4,208 distinct windows — 2.27 rows per window, matching concurrent users
Identity collision resolution	Passed	Two subjects sharing a username resolve to distinct devicequalified identities
Scorer idempotence	Passed after fix	Re-running the scorer no longer duplicates alerts (Section 5.5)

Check	Result	Interpretation
Activities syncer correctness	Passed after fix	Cross-machine contamination eliminated; 200 rows in 189 ms, previously over 120 s
ONNX deployment	Passed	All three graphs load and execute on the server against a real input batch
API endpoint availability	Passed	Four ML endpoints return stored predictions across three instances
Dashboard availability	Passed	pta.emotionflow.site returns HTTP 200 over TLS with current model results
Disk guard safety gate	Passed	Guard refuses to prune when any pipeline watermark is missing or ahead of the cutoff

5.1.3 Dataset and Splits The dataset covers 2 July to 18 July 2026: seventeen days across four subjects on four machines. Splits are per-user chronological at seventy, fifteen and fifteen per cent. A single global date cut was evaluated and rejected, because one subject stops producing data on 12 July and would therefore be absent from the test set entirely, making the test score unrepresentative of the cohort.

Table 8. Dataset composition and splits.

Feature table	Rows	Usable features	Train	Validation	Test
cheat_detection_features	90,654	200 of 273	63,456	13,599	13,599
unified_time_series	9,533	93 of 193	6,657	1,427	1,429
employee_growth_features	41	112 of 202	24	5	8

Figure 8 is a tall diagram and is published online at full resolution rather than reproduced here.

View figure (vector, zoomable) Download PNG

Figure 8. Per-user chronological train, validation and test split. Every subject appears in all three splits, and validation and test are always later in time than training.

5.2 Automation Testing

Nine defects were identified during this work, each found by measurement rather than by code reading. This section is reported in full because the defects share a property that shaped the project's testing approach: **every one of them passed conventional structural integrity checks.** No exception was raised, no error was logged, referential integrity held, and timestamps aligned correctly — while the output was wrong.

Table 10. Defects found and corrected.

#	Defect	How it was detected	Root cause	Correction
1	Cheat-feature aggregation stalled for 16 days	Watermark not advancing while raw data arrived	A component aggregate grouped at four-column grain but joined on one column, multiplying rows and violating uniqueness	Collapse the aggregate to the join grain using statistical mode for categorical columns
2	All subjects blended into single rows	Row count exactly equalled distinct window count across a four-person cohort	Aggregation ran globally and assigned identity afterwards	Per-device aggregation loop with staging tables; identity became an input

#	Defect	How it was detected	Root cause	Correction
3	Two people resolved to one identity	Two participants shared a Windows account name	Canonical identity derived from operating-system username alone	Device-qualified identity plus a manual override table
4	Growth pipeline produced nothing	Feature table empty despite weeks of source data	Three faults: a null watermark yielding zero loop iterations, a missing uniqueness constraint referenced by the insert, and a canonical-versus-raw identity mismatch in joins	Initialise the watermark, add the constraint, introduce canonical views
5	Cheat model at chance (part 1)	Training loss 0.27 against validation loss 3,700	Standard scaling collapsed on heavy-tailed counters; one feature reached 10,595 standard deviations	Signed logarithmic transform, then scaling, then clipping at ten standard deviations
6	Cheat model at chance (part 2)	ROC-AUC 0.52 after the scaling fix	Injection forced perfect regularity, but 22% of genuinely active windows already have zero interval variance — the task was unsolvable	Require volume and regularity jointly over contiguous two-minute episodes, across three independent vectors
7	Duplicate activity records	339,670 duplicates present despite a passing duplicate check	Concurrent syncer instances processed overlapping ranges; the check tested a key the duplicates did not violate	Device-scope each syncer instance
8	Syncer joined on a non-unique identifier	200 rows took over 120 seconds	Join used the operatingsystem process id, which is identical across machines, matching processes on different endpoints	Join on a globally unique identifier: 189 ms, a 635fold improvement, and cross-machine matching became structurally impossible
9	Automated pruning destroyed unaggregated data	Raw mouse data absent for a period before features were built	The disk guard pruned by age with no knowledge of pipeline progress	Prune only behind the minimum of all three watermarks; refuse to delete when any is missing, and archive regardless

Figure 9. Cheat-detection recovery path. Two independent defects each held the model at chance; correcting the scaling collapse alone was not sufficient, and the evaluation design had to be corrected as well.

Defects five and six together moved the model from 0.52, indistinguishable from chance, to 0.9558. Defect six is the more instructive: the model was correctly reporting that an unsolvable task could not be solved, and the fault lay in the evaluation, not the model. Defects one, two and four share the silent-success pattern — the system reports success while producing wrong or no output — which is why verification here asks whether output has the shape reality requires, not merely whether execution completed.

5.2.1 Testing Still Required

- Automated regression tests for the aggregation functions, currently verified by inspection and row-shape assertions rather than a fixed test corpus;
- Endpoint service crash-recovery tests: the process and focus services fail during 17 to 44 percent of active hours and the cause is not yet isolated;
- Load testing of the serving API under concurrent access;
- Negative testing of the disk guard against malformed and adversarial watermark states;
- Cheat-detection evaluation against real automation tools rather than synthetic injection;
- Bias and fairness evaluation across subjects, machine configurations and working patterns.

Chapter 6 Performance Evaluation

6.1 Environment and Evidence Boundary

All model results were produced against the collected dataset described in Section 5.3: seventeen days, four subjects, four machines, cut at 18 July 2026. Deployment measurements were recorded on the project server, a single Linux host running the database, pooler, API instances and web tier concurrently.

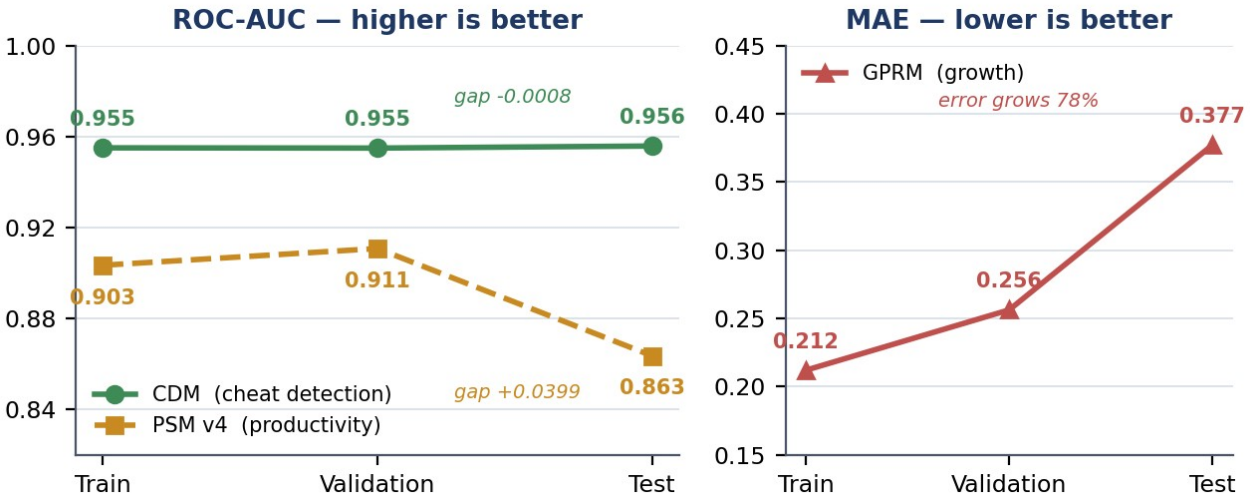
> **Evidence boundary.** These results establish measured behaviour on one cohort over one collection window. They do not establish population-level accuracy, performance under production load, robustness to a live adversary, or generalisation to a workforce with different roles or tooling. Every figure below should be read within that boundary.

6.2 Evaluation Design

Every model is compared against an explicit baseline it must beat, with the same metric reported on all three splits. This is what makes the chapter interpretable: a single test figure cannot distinguish a model that learned from one that memorised, whereas the train-to-test progression can. The baselines are prevalence for cheat detection, persistence for productivity, and the training mean for growth.

6.3 Model Results

Table 11.



Model generalisation across splits.

Model	Metric	Train	Validation	Test	Train → test gap
Cheat Detection (CDM)	ROC-AUC (higher better)	0.9550	0.9549	**0.9558**	**−0.0008**
Productivity (PSM)	ROC-AUC (higher better)	0.9033	0.9107	0.8635	+0.0399
Growth (GPRM)	MAE (lower better)	0.2119	0.2562	0.3772	+0.1653

Figure 10. The same metric on every split. CDM holds its performance on unseen data; PSM and GPRM degrade.

Table 12. Model performance against baselines.

Model	Split	Model score	Baseline	Winner
PSM	Train	0.9033	0.7919 (persistence)	Model
PSM	**Test**	0.8635	**0.8563**	**Model**

Model	Split	Model score	Baseline	Winner
GPRM	Train	0.2119	0.2612 (train mean)	Model
GPRM	**Test**	0.3772	**0.3649**	**Baseline**
CDM	Test	0.9558	0.0988 (prevalence, PR-AUC basis)	**Model**

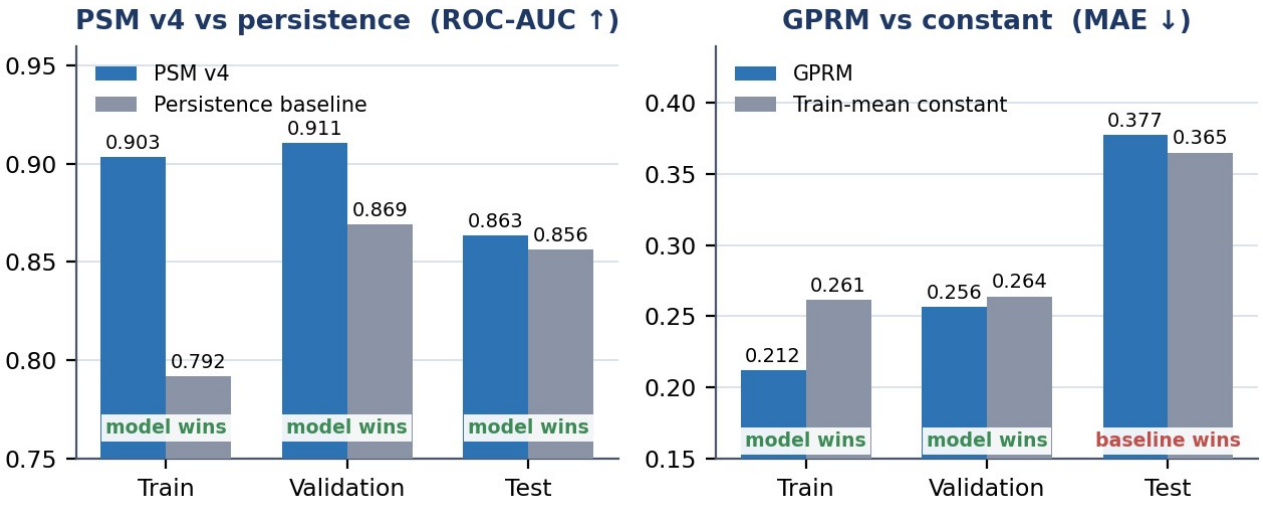


Figure 11. Each model against the baseline it must beat. Both cross from winning on training data to losing on test.

The pattern is unambiguous. Productivity and growth beat their baselines only on data they were fitted to — the signature of memorisation, not learning. Cheat detection beats its reference on every split, which is what generalisation looks like.

6.3.1 Cheat detection in detail

Table 13. Cheat-detection performance overall and per injected vector.

Metric / vector	Test value	Interpretation
Overall performance		
ROC-AUC	0.9558	Ranks anomalous windows above normal ones reliably
PR-AUC	0.6377	Against a random baseline of 0.0988 — a 6.5-fold lift
Recall	0.8810	Detects roughly nine in ten injected automation windows
Precision	0.5676	Approximately 43% of flagged windows are false positives
F1	0.6904	At the F1-optimal operating threshold
Per injected vector (ROCAUC / recall)		
bot_rhythm	0.9629 / 0.9236	Constant-interval synthetic input
replay_macro	0.9635 / 0.9205	Repeated recorded action sequence
burst_injection	0.9808 / 0.9773	High-volume regular bursts

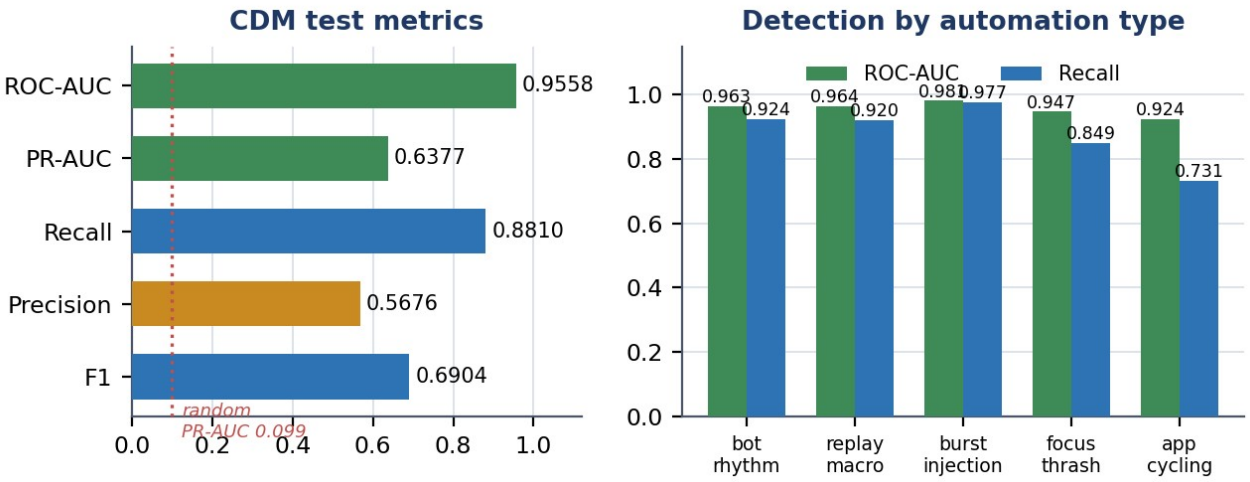


Figure 12. CDM test performance, and detection consistency across three independently constructed automation vectors.

Consistency across three independently constructed vectors (0.9629–0.9808) is the evidence that the model learned a general notion of automation, not an artefact of one injection method.

> **Operational caveat.** Precision of 0.568 means more than four in ten flagged windows are false positives at this threshold. The output is therefore a triage signal directing human attention, not a verdict. This is consistent with the false-alarm limitation the literature identifies as the principal barrier to operational insider-threat detection [12], [16].

6.3.2 Productivity A capacity sweep across model dimensions and dropout rates was run, and the productivity model was then rebuilt as a five-seed ensemble with a per-user-relative engagement target. It beats the persistence baseline on all three splits — 0.8635 against 0.8563 on test — though the margin is narrow. The result is therefore reported as a genuine but slender positive rather than a decisive one.

Two causes are identifiable. First, the dataset provides 6,657 training sequences from four subjects over seventeen days, which is small for an attention-based architecture. Second, and structurally more important, the literature associates the highest productivity-prediction accuracy with hybrid models incorporating physiological and environmental signals, reporting 81 per cent for inputbased approaches against 96 per cent for hybrid ones [3], [6]. This project has neither modality. The productivity model is operating on second-generation data with a third-generation architecture, and the outcome is consistent with that mismatch.

6.3.3 Growth The growth model was trained on 24 daily rows, and its intended thirty-day context window exceeds the entire collection period, so the architecture could not be exercised as designed. It memorises the training days and loses to a constant predictor on test. This is an integration result — the architecture implements, trains, exports and executes — not a forecasting capability.

Growth therefore ships as the rule-based engine, which produces four distinct, evidence-backed assessments across the four subjects, each traceable to named signals. The engine reports its own confidence from observed history and declines to score below four days. It makes no accuracy claim, because no ground truth for growth exists; its defence is transparency rather than validation, which directly addresses the explainability requirement the literature identifies as a precondition for adoption in this domain [5], [27].

6.4 Engineering Performance

Table 14. Recorded runtime measurements.

Measurement	Recorded result	Qualification
Activities syncer join, 200 rows	189 ms	Previously over 120 s on the same data — a 635fold improvement after the join-key correction
Deployed model runtime footprint	152 MB	Three ONNX graphs with preprocessing parameters and manifest, against roughly 2 GB for a full framework
Cheat model artefact	722 KB	TranAD exported to ONNX

Measurement	Recorded result	Qualification
Productivity model artefact	6.7 MB	Temporal fusion architecture exported to ONNX
Growth model artefact	82 KB	Dilated TCN exported to ONNX
Database size	88 GB	Recorded on the project server at the time of writing
Base tables	50	Raw events, identity, features, watermarks and predictions
Server volume utilisation	114 GB of 145 GB (79%)	Under automated disk-guard management with offsite archival
Predictions written	3,466	3,000 cheat scores, 31 alerts, 394 productivity, 41 growth — all version-tagged

Serving-latency figures under concurrent load are not reported because no load test was conducted. Scoring currently runs on demand rather than on a schedule, so predictions are current as of the last invoked run.

6.5 Accuracy and Validity Boundary

Stated explicitly, because these are the claims the evidence does not support:

- **No productivity accuracy claim.** No human productivity annotations exist. The productivity model is evaluated against a behavioural proxy and loses to persistence on unseen data.
- **No growth-forecasting claim.** Twenty-four training days cannot fit a temporal forecaster. The rule engine is transparent, not validated against outcomes.
- **Cheat-detection metrics come from synthetic anomalies.** They measure sensitivity to automation-like deviation from learned normal behaviour, not real-world prevalence, and cannot cover an evasion strategy that was not anticipated.
- **Four subjects over seventeen days.** Normal behaviour is learned from a small cohort with similar roles and tooling; a different workforce may present a different normal.
- **Known data-quality faults remain.** The process and focus services crash during 17 to 44 per cent of active hours; one aggregated column reports implausible values and is excluded from the growth engine; several intended growth signals are never populated; and the capture-layer productivity heuristic classifies all browser time as productive.

One earlier project document assessed data quality at 8.9 out of 10 on structural criteria. That assessment is withdrawn. Every defect in Table 10 satisfied those structural criteria while producing wrong output, which demonstrates that structural validity is necessary and not sufficient. No replacement single-figure score is offered, because reducing data quality to one number is what produced the misleading claim.

Chapter 7 Business Model

7.1 Market Research

- Organizations with distributed engineering or knowledge-work teams that need behavioural measurement beyond time tracking;
- Security and compliance teams requiring endpoint behavioural signal to complement networklevel controls, addressing the endpoint-network disconnect identified in Section 2.7;
- Outsourcing and managed-service providers who must evidence that billed work was performed by a person;
- Research groups requiring an instrumented behavioural telemetry platform.

High-impact applications — disciplinary process, hiring, termination, performance appraisal and law enforcement — are excluded without independent validation, governance and human oversight, for the reasons given in Section 3.10.

7.2 Unique Value Proposition (UVP)

The proposed value is not that the system measures productivity accurately; Section 6.5 states that it does not yet demonstrate this. It is that the system provides an auditable behavioural record at a resolution sufficient

to distinguish human from automated activity, with every conclusion traceable to the signals that produced it. The differentiators to validate are the multi-granularity feature layer, the transparency of the growth engine, and the absence of content capture, which materially lowers the privacy and compliance burden relative to screenshot-based tools.

7.3 Targeted Audience

PTA is an academic engineering prototype. It has no billing, subscription, payment or tenancy implementation, and no commercial pilot has been conducted. The following is a set of hypotheses to validate, not evidence of traction.

7.4 Monetization Strategy

- a self-hosted institutional licence with installation and support, which suits the privacysensitive segment;
- a hosted per-seat service, contingent on multi-tenancy and data-isolation work that is not implemented;
- an integrity-verification service billed per audited endpoint, drawing on the one model with demonstrated skill;
- the feature-engineering layer licensed as a component to existing workforce-analytics vendors.

No pricing is proposed. Server cost, storage growth — 88 GB from four endpoints over the collection period — support burden and compliance cost have not been measured, and a price set without those figures would be arbitrary.

7.6 Propose Business Plan

Table 15. Lean validation plan.

Hypothesis	Minimum validation evidence
Organisations need automation detection, not just time tracking	Five or more structured interviews with security or delivery leads, plus one paid pilot
Absence of content capture is a purchasing differentiator	Procurement or compliance interviews at privacysensitive organisations
The growth assessment is understood and accepted by employees	Usability testing with defined comprehension tasks and a reactance measure
Detection performance holds against real evasion tools	Evaluation against commercially available jigglers and automation scripts
Productivity scoring can reach usable accuracy	Sixty-day collection with a labelled subset and a documented annotation protocol
Customers will pay	Written pilot commitment, not survey interest

7.10 Legal Considerations

Table 16. Risks and mitigations.

Risk	Mitigation
Employee rejection and reactance, which the literature identifies as the dominant failure mode [11], [34], [35]	No content capture; transparent per-signal evidence; employee-facing disclosure; engagement weighted at zero so the score cannot reward overwork
False positives from the cheat model damaging trust in an individual case	Present as triage with the measured 46% false-positive rate disclosed; require human review; never automate consequence
Regulatory exposure under workplace privacy and data-protection law	Implement retention limits, deletion, anonymisation and access logging before any deployment outside a consenting research cohort
Model degradation as work patterns or tooling change	Versioned prediction tables allow retraining and side-by-side comparison without schema change
Endpoint capture reliability	Services currently fail during 17–44% of active hours; supervision and crash recovery are the first priority in Chapter 8
Scope creep toward high-impact decisions	Responsible-use constraints stated in the product interface, not only in documentation

7.10.1 SDG Alignment The project has relevance to Sustainable Development Goal 8, decent work and economic growth, and Goal 9, industry, innovation and infrastructure. A cautious interpretation is that transparent, evidence-linked measurement without content surveillance is a less harmful form of workplace monitoring than the screenshot-based alternatives it competes with, and that the growth orientation addresses a documented deficit bias in workforce analytics. No field trial has been conducted, and no claim of realised social impact, improved worker wellbeing or organisational adoption is made.

Chapter 8 Future Directions and Conclusion

Evidential and operational risk prioritise work to be addressed in the future.

1. Improve the reliability of endpoint captures. During 17 to 44 per of the process and focus services, failure occurs. A 100% active hour per machine. This is the largest source of missing data and Thus it is a constraint on each result of the downstream. Identify the root cause from launcher logs, and add Supervised restart with backoff and instrument uptime as first class metrics.
2. Take two months of data.Gather twelve two months of data. The same validation error signature is seen in both failed models: As training error decreases, rising. About 60 days for every subject would be the equivalent time. Since the grow model is supposed to need 30 days for the 30-day context, 240 rows are used daily and the 30-day context will be usable for the first time after 240 days. time. This is the only change that would affect the results of Chapter 6 the most.
3. Fix capture-layer productivity heuristic that makes all browser activity equal toproductive in any content, and aggregation defect with implausible fatigue.Currently counts that are used to exclude a signal from the growth engine.
4. Evaluate cheat detection against real evasion tools. Current metrics come from synthetic injection. Testing on commercially available mouse jigglers/automation scripts. necessary prior to any claim of operation.
5. 2.3 Privacy-by-design controls not included in the solution: Implementation: local filtering of sensitive applications prior to capture, identity, Pseudonymisation in the analytical layer, edge-side feature extraction to decrease transmission, and a retention and deletion policy.
6. Add endpoint integrity protection: cryptographic signing of telemetry before it leaves the either of these, which the literature considers prerequisites for any machine, are treated as separate in the literature.Tamper detection and machine is treated separately in the literature. adversarial deployment.
7. Get ready to score on the server. Currently inference is run as a demand-based process; a scheduled job would Make predictions that are up to date and does not require any code changing.
8. Create an employee perspective. The literature indicates transparency of the monitored. Acceptance was the primary determinant and person was the most prominent influence. The rule engine can already create per-signal There is evidence adequate for this; at present only researchers can view this.
9. Add explainability to the neural models. The attribution of the inputs to the cheat model would Let a reviewer know which behavioral signals were used to trigger a flag and which the current reconstruction used. score doesn't reveal.
10. Perform bias and fairness assessment for roles, machine setup and working Patterns – whether the system systematically disadvantages any working style.
11. Extend beyond Windows. The product will only work on macOS and Linux endpoints, restricting use. To homogeneous environments, a system.
12. Test, stress and load test the serving tier, add multi-tenant isolation and practice recovery.

    Not until a frozen release, before claiming production readiness.

8.1 Conclusion

This project addressed three gaps from a systematic review of forty papers: monitoring that detects statistical outliers without inferring intent, workforce analytics oriented to deficit rather than development, and endpoint behaviour invisible to network-level security. It responded with a complete deployed implementation — five capture services, a central database with a multigranularity feature-engineering layer, a canonical identity layer, three model architectures, a serving API and a public dashboard.

The strongest supported conclusion is that **two of the three models work and one does not, and the evidence identifies which.** Cheat detection achieves ROC-AUC 0.9558 on held-out data, a train-to-test gap of 0.0008 and a 6.5-fold lift over its random baseline, consistently across three independent automation vectors.

The growth model beats its baseline only on data it was fitted to; the productivity model beats persistence on all three splits, though only narrowly. That is reported directly, with its causes: insufficient longitudinal data — decisive for growth and the reason productivity's margin stays slender — and for productivity the absence of the physiological and environmental modalities the literature ties to the accuracy gains this architecture was designed to exploit.

The engineering contribution is arguably larger than the modelling one. Nine defects were found by measurement, each passing structural integrity checks while producing wrong output — feature tables blending four people into single rows, a pipeline reporting success while processing nothing, two people resolving to one identity, an evaluation so ill-posed the task was unsolvable. After two corrections, detection rose from "chance" to 0.9558. The lesson: structural The usefulness is not sufficient, but necessary, and the useful question is not whether execution completed But is output shaped as the reality demands.

If a capability was not validated, the system ships the honest alternative to the capability, instead of the unvalidated one. impressive one. A clear rule-based engine provides a verdict for every case, enhancing growth. the signal name, the recent value, the baseline value and the effect size, an unvalidated neural measure — are displayed.The signal name, the recent value, the baseline value, and the effect size, an unvalidated neural measure, are displayed. A score that precedes any user would be indefensible and the explainability that this gives would be itself an explainable. The precondition that is referred to in the literature in this domain.

PTA is a working, measured research prototype, therefore, with one validated capability, showed data-engineering base, and a declared limit to what its evidence supports. The way forward is clear – improve the capture reliability and expand collections to two. evaluate and implement privacy controls for the review, in months, against actual evasion tools. identified. Having a clear boundary, not ambiguous, leaves a foundation a The next stage can be developed upon

Appendix A - User Manual

A.1 Deploying the Tracker

1. Check that the participant has given informed consent and is aware of what is being recorded: No content of any kind, plus timing, counts and window metadata.
2. Copy the configuration file (with the server connection information) next to the launcher.
3. Launch the launcher, which manages all five capture services.
4. Check the arrival of the telemetry data by asking for recent data for that device on the server.

A.2 Running the Feature Pipelines

1. Each of the aggregation functions can be called as an individual function and will only operate on the interval since its call was made. watermark.
2. Functions are safe to rerun: They are watermark-driven, with conflict handling, so they are safe to rerun. invocation neither duplicates nor skips data.
3. Check for progress by checking the watermark table, if the watermark has stalled then the check is an. It's not that there's no data but that there is no aggregation failure.

A.3 Training and Deploying Models

1. The 33 phase scripts are executed sequentially, and they can be run independently and will each produce a report of their activities outputs.
2. When training, model artefacts are also generated along with the parameters of the preprocessing. training split. These will have to go together — a model without its transform is not reproducible.
3. Export to ONNX, copy artefacts+manifest to server and run the deployment. The verification script loads each graph and runs it against a "real batch.

A.4 Reading the Dashboard

The results page shows train, validation and test metrics side by side for each model, with the applicable baseline. If there is a difference between train and test before the headline figure If the score for the training is high, but the large gap is still a positive score, it means that the person has memorised enough. The growth panel Demonstrates the evaluation of each subject and its corresponding signals. The methodology page documents splits, preprocessing, design and evaluation protocol for injection.

Goto the pta.emotionflow.site website in the browser. There is no need to register and it is a readonly interface.

A.5 Responsible Use

IBM PTA is not designed to be used as a lie detector, a productivity verdict, a disciplinary tool or as the only will not be used as a basis for employment, security or academic decision. The cheat score is a triage signal with Its operating threshold has an estimated false positive rate of about 46 per cent. Obtain informed consent Before installing the tracker on another individual's computer, and explaining what is being recorded and how long it is retained.

Appendix B - Software Requirements Specification (SRS)

B.1 Introduction

B.1.1 Purpose

The purpose of this document is to define the Software Requirements Specification (SRS) for the Enterprise Productivity Tracking System (EPTS), also referred to as PTA 2.0. This document serves as the primary blueprint for the development team, detailing the functional and nonfunctional requirements, system architecture, and operating environment to ensure the successful delivery of an AI-powered workforce analytics platform.

B.1.2 Scope

EPTS is designed to replace traditional, easily bypassed time-tracking software with intelligent, AI-driven behavioral analytics. The scope encompasses:

- **Data Collection:** Deploying a lightweight, Rust-based desktop agent that captures real-time telemetry across five domains (keyboard, mouse, process, active window focus, and resource usage) without disrupting the employee's workflow.
- **Data Aggregation:** Utilizing a centralized PostgreSQL database with independent connection pools to process approximately 73,700 raw events per user, per day.
- **Machine Learning Integration:** Employing ONNX-based ML models to instantly detect artificial inputs/evasion tactics (Cheat Detection), generate role-adaptive productivity scores, and forecast long-term employee growth trajectories.
- **Dashboards:** Providing secure, web-based interfaces for both administrators and employees to view performance insights and skill gap recommendations.

B.1.3 Definitions

- **Telemetry:** The automated recording and transmission of data from remote desktop agents to a centralized IT system.
- **Mouse Jiggler:** Hardware or software used to simulate mouse movement, artificially keeping a computer active.
- **Watermark Gap-Skip:** An optimization technique used in the aggregation pipeline to process only new data since the last successful read.

B.1.4 Acronyms and Abbreviations

- **EPTS:** Enterprise Productivity Tracking System
- **PTA:** Productivity Tracking Agent
- **API:** Application Programming Interface **ML:** Machine Learning
- **ONNX:** Open Neural Network Exchange
- **WBS:** Work Breakdown Structure
- **ETW:** Event Tracing for Windows
- **SPOF:** Single Point of Failure

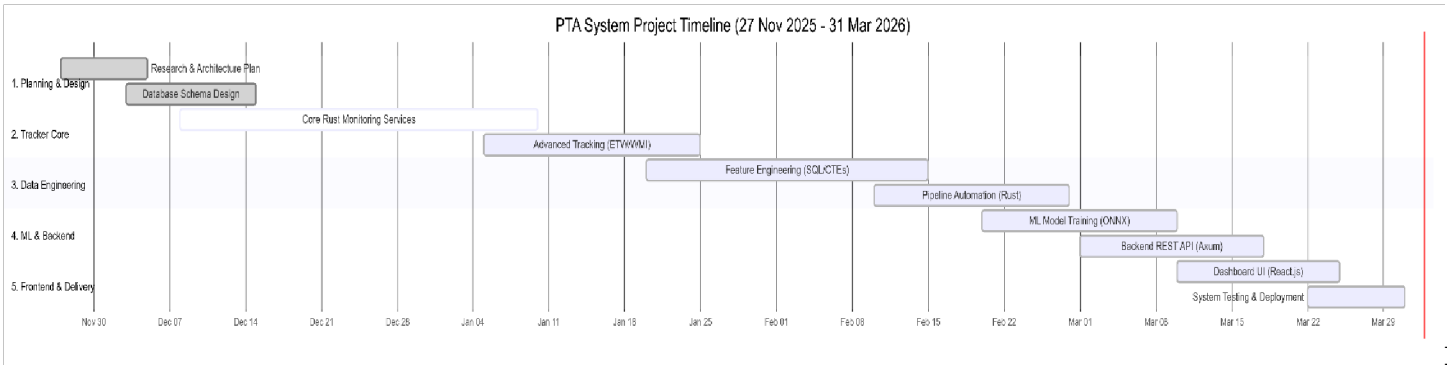
B.2. Project Planning and Management

B.2.1 SWOT Analysis

Strengths	Weaknesses
High-Performance Stack: Rust-based microservices ensure <2% CPU usage. **Deep Telemetry:** Captures micro-behavioral data rather than superficial metrics. **Hybrid Architecture:** Blends event-driven and polling methods for maximum efficiency.	**Platform Dependency:** Initial tracker relies heavily on Win32/ETW APIs, restricting it to Windows OS. **Data Volume:** High data generation requires substantial database storage scaling.

Opportunities	Threats
Market Demand: Growing need for accurate, cheat-proof remote work analytics. **AI Integration:** Positioning as an AI-coach rather than a strict surveillance tool appeals to modern HR departments.	**Privacy Regulations:** Must strictly comply with data minimization and employee privacy laws (e.g., GDPR). **Adversarial Evolution:** Cheat software constantly evolves, requiring continuous ML model updates.

B.2.2 Gantt Chart



Work Breakdown Structure (WBS)

1. **System Orchestration & Collection:**
   (a) 1.1 PTA Launcher (Orchestrator)
   1.2 Keystroke Monitoring Service
   1.3 Mouse Input Monitoring Service
   1.4 Process Monitoring Service
   1.5 Window Focus Monitoring Service
   1.6 Activities Monitoring Service
2. **Data Pipeline & Storage:**
   (a) 2.1 PostgreSQL Database Setup
   2.2 Unified Time Series (UTS) Aggregator
   2.3 Cheat Features Aggregator 2.4 Growth Features Aggregator
3. **Machine Learning & Backend:**
   (a) 3.1 Backend API Development
   3.2 Productivity Scoring Model (Inference)
   3.3 Cheat Detection Model (Inference)
   3.4 Growth Prediction & Recommendation Model (Inference)
4. **Frontend & Dashboards:**

(a) 4.1 Admin Analytics Dashboard

4.2 Employee Insights Dashboard

B.3 Overall Description

B.3.1 Product Perspective

EPTS operates as a distributed client-server application. It consists of a local client component (orchestrated microservices running on the employee's machine) communicating with a centralized backend and database infrastructure. It acts as an independent system but is designed to provide data outputs that could eventually be integrated into broader HR or ERP systems.

B.3.2 Product Function

- **Passive Monitoring:** Operates silently in the background capturing 5-domain telemetry.
- **Real-time Feature Engineering:** Compresses raw events into time-series features (per minute, per 5 seconds, per day).
- **Anomaly Detection:** Flags irregular input patterns characteristic of scripts, macros, and hardware jigglers.
- **Productivity Evaluation:** Calculates focus depth and work output without reading actual keystroke content.
- **Talent Forecasting:** Analyzes historical data to predict skill growth and recommend improvements.

3.3 Operating Environment

- **Client (Tracker Agent):** Windows 10/11 (x86_64 architecture). Requires <200 MB RAM and <2% CPU. Standard user permissions required, with Admin rights for deep ETW process monitoring.
- **Server (Backend):** Linux environment recommended. Requires PostgreSQL 14+, ONNX Runtime, and a Node.js/Python backend.
- **Network:** Requires outbound TCP access on standard database/API ports (e.g., 5432, 443).

B.3.4 Design and Implementation Constraints

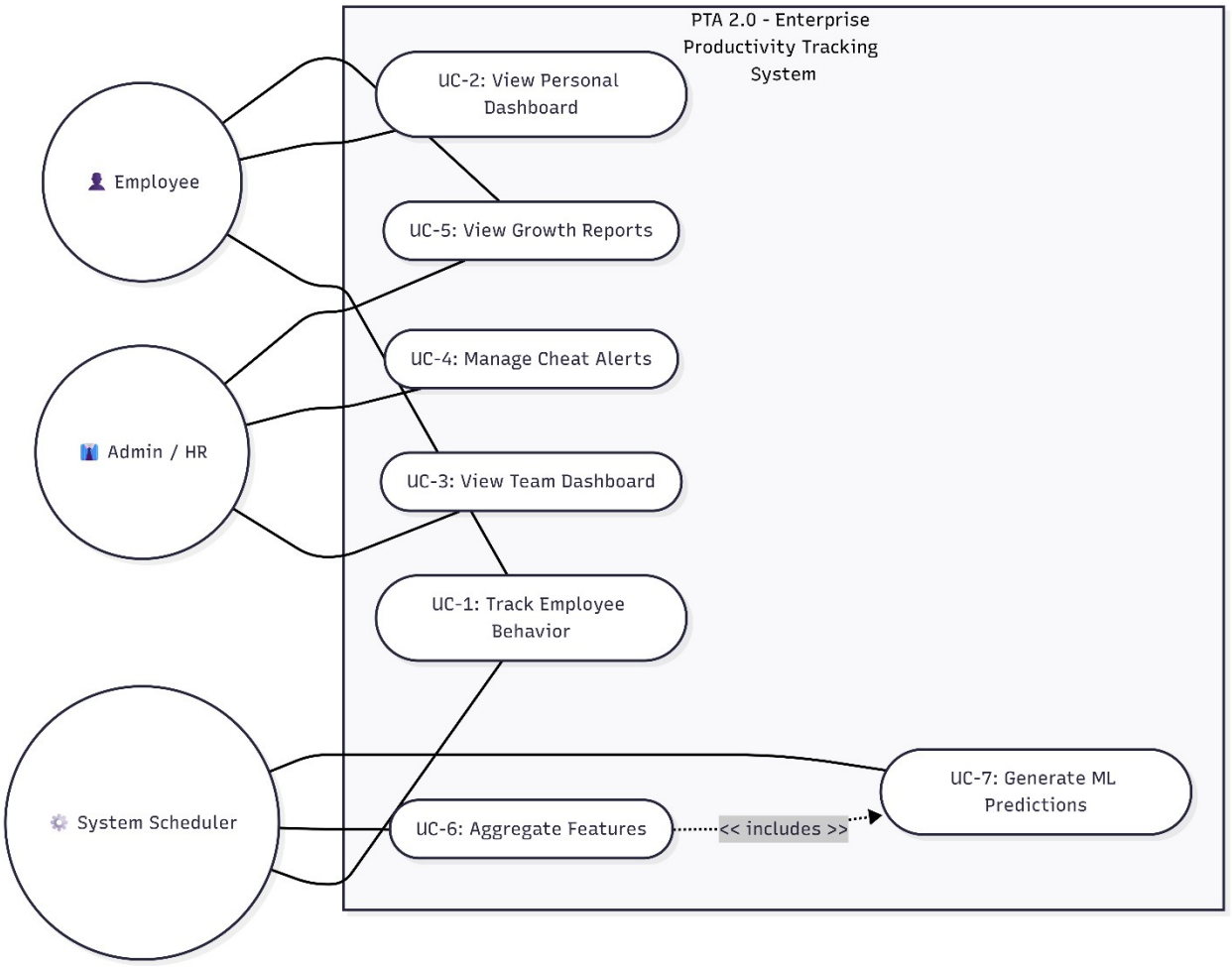
- **Language Restriction:** All core tracker agents must be written in Rust to ensure memory safety and zero-overhead performance.
- **Database Exclusivity:** PostgreSQL is the sole data store; no secondary caching databases (like Redis) will be utilized to maintain architectural simplicity.
- **Privacy Constraint:** The system must under no circumstances log the actual content of keystrokes (no keylogging of passwords or messages); it must only log the *timing* and *frequency* of events.

B.3.5 Assumptions and Dependencies

- **Assumption:** The target employee workstations will have a relatively stable internet connection to flush data to the PostgreSQL database.
- **Dependency:** The machine learning models depend on the ONNX runtime library being successfully integrated into the backend API.

4. Requirement Identifying Technique

4.1 Use Case Diagram



4.2 Use Case Description

Use Case ID	Name	Actor	Description
UC-01	Track Telemetry	System	The PTA Launcher spawns 5 microservices to passively collect behavioral data and stream it to the database.
UC-02	Detect Evasion	System	ML engine analyzes 5-second mi-crowindows to flag non-human inputs.
UC-03	View Analytics	Admin	Administrator accesses the dashboard to view aggregated productivity scores and cheat alerts for the organization.

Use Case ID	Name	Actor	Description
UC-04	View Personal Growth	Employee	Employee views their own dashboard to see predicted growth trajectories and system-recommended upskilling paths.

5. Non-Functional Requirements

5.1 Performance Requirements

- The tracker agents shall consume no more than 2% of CPU resources during normal operation.
- The system must handle the ingestion of up to 73,700 events per user per day without bottlenecking.
- Database aggregation queries (UTS, Cheat) must execute in under 2 seconds.

B.5.2 Safety Requirements

- If CPU usage spikes above 5% due to the tracker, the agent must automatically throttle its polling frequency or pause tracking to prevent system crashes.
- In the event of network failure, the tracker must cache telemetry locally or implement exponential backoff to prevent network flooding upon reconnection.

B.5.3 Security Requirements

- All data transmitted between the tracker and the PostgreSQL database must be encrypted via TLS.
- Access to the frontend dashboards and API endpoints must require strict role-based authentication (JWT or similar).
- SQL queries must be parameterized to prevent SQL injection attacks.

B.5.4 Software Quality Attributes

- **Reliability:** The system must achieve 99.5% uptime. The PTA Launcher must auto-restart any child microservice that crashes.
- **Accuracy:** The ML Cheat Detection model must achieve a precision rate of 85% to minimize false accusations.

B.5.5 Business Rules

- Employees must explicitly consent to monitoring before the PTA agent is installed or activated on their machine.
- Productivity scores cannot be the sole metric used for employee termination without human review.

5.6 Interoperability

- The system utilizes a central PostgreSQL database, allowing future interoperability with business intelligence tools (like Tableau or PowerBI) via standard ODBC/JDBC connections. **B.5.7 Extensibility**
- The microservice architecture ensures that new trackers (e.g., a webcam posture tracker) can be added as an independent Rust service without altering existing codebases.

B.5.8 Maintainability

Code shall be highly modularized. Rust services will implement comprehensive error logging to specific .log files for straightforward debugging.

B.5.9 Portability

While the tracker agents are Windows-specific (due to Win32 APIs), the Backend API, PostgreSQL database, and Frontend Dashboard are fully platform-agnostic and can be deployed on Windows, Linux, or macOS servers.

B.5.10 Reusability

The ONNX ML models generated for this project can be reused or retrained on different datasets without modifying the core backend inference logic.

B.5.11 Installation

- The client installation shall consist of a single executable installer that sets up the PTA Launcher and its accompanying Rust microservices.
- Server deployment shall utilize Docker containers for streamlined setup of the database and API.

6. Other Requirements

6.1 On-line User Documentation and help System Requirements

The system will provide an interactive onboarding guide within the dashboard to explain to employees how their productivity score is calculated, ensuring transparency.

B.6.2 Purchased Requirements

No specialized hardware purchases are required. The system utilizes open-source libraries (Rust crates, ONNX, PostgreSQL). Cloud hosting (e.g., AWS/Azure) will be required for the production database.

B.6.3 Licensing Requirements

The software will utilize open-source frameworks but the core EPTS proprietary logic will remain closed-source for enterprise deployment.

B.6.4 Legal, copyright and other Notices

The system strictly adheres to data minimization principles. It logs behavioral timings and application IDs, but not the contents of private communications, protecting against intellectual property or privacy breaches.

B.6.5 Applicable Standards

- **IEEE 830-1998:** Recommended Practice for Software Requirements Specifications.
- **OWASP Top 10:** Security guidelines for the web dashboard and API.

B.6.6 Reports

- **Executive Summary:** Weekly aggregated report of department-wide productivity and anomaly flags.
- **Employee Feedback Report:** Monthly automated report highlighting an employee's strengths, weaknesses, and skill growth predictions.

B.7 References

1. IEEE Std 830-1998, ”IEEE Recommended Practice for Software Requirements Specifications.”
2. ISO/IEC 25010:2011, ”Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE).”
3. OWASP Foundation, ”OWASP Top 10 Web Application Security Risks,” 2021.
4. Rust Programming Language, ”The Rust Reference,” 2024.
5. PostgreSQL Global Development Group, ”PostgreSQL 14 Documentation,” 2024.

Appendix C - Software Design Description

C.1. Introduction

C.1.1 Purpose This Software Design Description (SDD) document provides a comprehensive technical design specification for the Productivity Tracking & Analytics (PTA) system. The document describes the system's architecture, design methodology, data models, data engineering pipelines, and module-level design decisions. It serves as a reference for developers, evaluators, and stakeholders to understand how the system is constructed and how its components interact.

C.1.2 Scope The PTA system is an AI-powered employee monitoring and productivity analytics platform designed to:

- Track employee desktop activity in real-time through 6 specialized monitoring services.
- Collect multi-modal behavioral data: process execution, mouse input, keyboard input, focus patterns, and user activities.
- Store raw events across 27 PostgreSQL tables (1,235+ columns).
- Aggregate raw data through 3 data engineering pipelines into ML-ready feature tables (746 features).
- Provide data infrastructure for 3 machine learning models: Productivity Scoring, Cheat Detection, and Employee Growth Prediction.

C.1.3 Project Background In modern remote and hybrid work environments, organizations need data-driven insights into employee productivity, work patterns, and potential integrity issues. Existing solutions like Hubstaff, Time Doctor, and ActivTrak rely on simplistic metrics (screenshots, URL logging) and lack deep behavioral analysis. The PTA system addresses this gap by capturing granular, multimodal behavioral data at sub-second resolution and engineering it into 746 ML-ready features for advanced analytics.

C.1.4 Motivation The motivation behind this project stems from several key observations:

1. **Remote Work Accountability Gap:** Employers lack visibility into how work hours are actually spent in remote setups.
2. **Limitations of Existing Tools:** Current monitoring tools capture surface-level metrics rather than deep behavioral patterns and are easily bypassed by mouse jigglers.
3. **AI/ML Potential Untapped:** No mainstream tool applies machine learning to detect productivity patterns, cheating behavior, or predict employee growth trajectories.
4. **Data Engineering Gap:** Raw event streams are useless without proper aggregation—a gap this project bridges with 3 dedicated pipelines.

C.1.5 Project Objective The primary objectives of the PTA system are:

1. **Build a high-performance multi-service desktop tracker** in Rust that captures microbehavioral inputs in real-time.
2. **Design a comprehensive relational database schema** to store multi-modal behavioral events securely.
3. **Implement 3 data engineering pipelines** that aggregate raw events into Unified Time Series, Cheat Detection, and Employee Growth feature tables.
4. **Generate a training dataset** of 746 engineered features suitable for training deep learning models.
5. **Prepare infrastructure** for backend API, ML model deployment, and intuitive frontend dashboard visualization.

C.2. Design Methodology and Software Process Model

C.2.1 Design Methodology The PTA system follows a **Domain-Driven Design (DDD)** methodology where each monitoring domain (process, mouse, keyboard, focus, activities) is a self-contained bounded context with its own:

- Data model (dedicated database tables)
- Service logic (independent Rust service binary)

- Event schema (domain-specific event structures)
- Aggregation rules (domain-specific feature extraction)

Key Design Principles:

- **Separation of Concerns:** Each monitoring domain is an independent service with its own tables.
- **Fail-Fast Architecture:** Each service operates independently — a crash in one does not affect others.
- **Data Immutability:** Raw event tables are append-only; no updates or deletes are permitted.

C.2.2 Design Pattern (Creational, Structural, Behavioral) The system leverages several design patterns to ensure scalability and maintainability.

Creational Patterns:

- **Builder Pattern:** Used in constructing complex database query objects and event structs. *Reasoning:* Process events have 46 columns; the builder pattern allows constructing events step-by-step without requiring massive parameters.
- **Factory Pattern:** Used by the pta-launcher to instantiate 6 different service types based on configurations.

Structural Patterns:

- **Repository Pattern:** Used for database access across all services. *Reasoning:* Abstracts SQL queries behind typed Rust functions, separating business logic from persistence logic.
- **Facade Pattern:** The pta-launcher acts as a facade over the 6 microservices, hiding process spawning, health monitoring, and shutdown complexities.

Behavioral Patterns:

- **Observer Pattern:** Used in the mouse and keyboard services. *Reasoning:* Uses Windows hook-based event observation. When input occurs, the OS notifies the service, avoiding highfrequency CPU polling.
- **Strategy Pattern:** Used in the 3 aggregation pipelines (UTS, Cheat, Growth) which share an execution structure but utilize different SQL feature computation strategies.

C.2.3 Software Process Models The PTA system follows a **Hybrid Agile + Incremental** software process model.

> **Why Hybrid Agile?** The system was built incrementally in phases (tracker services → database design → data engineering pipelines). Each component went through iterative refinement based on emerging data patterns (e.g., discovering the need for BIGINT columns after detecting 46 billion context switches). Continuous Integration and testing validated realworld tracking metrics before moving to the ML and web module phases.

C.3. System Overview

C.3.1 Architectural Design The PTA system implements a **Hybrid Distributed Architecture** combining:

1. **Event-Driven Architecture (EDA):** The mouse and keyboard services use OS hooks (SetWindowsHookExW) to instantly react to user input with zero-wait callbacks, capturing ~148 events/sec without CPU polling.
2. **Optimized Polling Architecture:** The process monitoring service intelligently polls Windows WMI/ETW APIs every 3 seconds, capturing only delta changes (resource spikes, new threads) to conserve memory.
3. **Pipeline Architecture:** Data engineering pipelines run inside the process monitor on timed intervals (60s and 1h), executing sequential CTE transformations to generate ML features. **C.3.2 Process Flow (Functional Requirement)**

System Startup & Tracking Flow:

1. User launches pta-launcher.exe.
2. Launcher spawns 6 independent microservices (Process, Mouse Backend, Mouse Agent, Keys, Focus, Activities).

3. All services establish their own independent connection pool (sqlx::PgPool) to the PostgreSQL database.
4. Services silently capture telemetry and perform batch INSERT operations into 20 raw event tables.
5. In the background, three distinct Data Aggregators execute SQL functions (populate_unified_time_series(),populate_cl rowth_features()) to condense millions of raw events into ML-ready feature tables.

C.4. Project Architecture

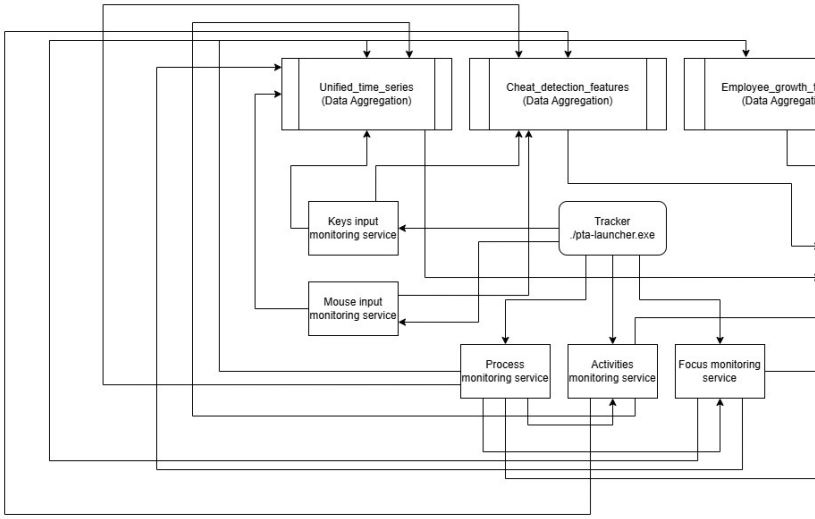
C.4.1 Desktop Tracker & Web Module

C.4.1.1 Functions (UI Designing - Figma, Canva) (UX based methodology) The PTA system bridges a high-performance desktop tracker with a web-based visualization frontend.

- **Tracker Module:** Operates entirely in the background (CLI-based) requiring zero user interaction to maintain authentic workflow environments.
- **Web UI (Dashboards):** The web module design follows a UX-based methodology

  (prototyped via Figma). The dashboards provide traffic-light color coding (green/yellow/red) for instant anomaly visualization. Admin views feature aggregated team scores, while Employee views focus on personalized upskilling paths and growth trajectories derived from the backend APIs.

C.4.1.2 Architecture (Box and Line Diagram)



C.4.2 Database module (Justification), Type, DDL (Data Definition Table) Database Justification & Type: The system utilizes **PostgreSQL 16** (Relational RDBMS).

Reasoning: Advanced SQL features (CTEs, Window Functions) are heavily required for the 3 aggregation pipelines computing 746 ML features. Its native UUID generation, robust MVCC (Multi-Version Concurrency Control) for 6 parallel writers, and structured JSONB support make it superior to NoSQL alternatives for this highly relational event schema.

DDL Summary (29 tables, 1,556 total columns):

- **Process Domain (10 Tables):** process_events, process_resource_history, process_performance_metrics, tracking CPU peaks, memory utilization, and application classifications.
- **Mouse Domain (6 Tables):** mouse_events tracking delta_x, delta_y, velocities, click precision, and drag-and-drop actions.
- **Keyboard Domain (4 Tables):** key_events capturing key hold durations and shortcut patterns without logging actual strings (Privacy compliant).
- **Focus & Activity (2 Tables):** Capturing window titles, context switches, and depth-of-focus duration.
- **ML Features (3 Tables):** unified_time_series, cheat_detection_features, employee_growth_features.

C.4.3 Administration module The administration and data engineering module forms the analytical core of the system. It processes raw data to be viewed securely by Admins (HR/Managers).

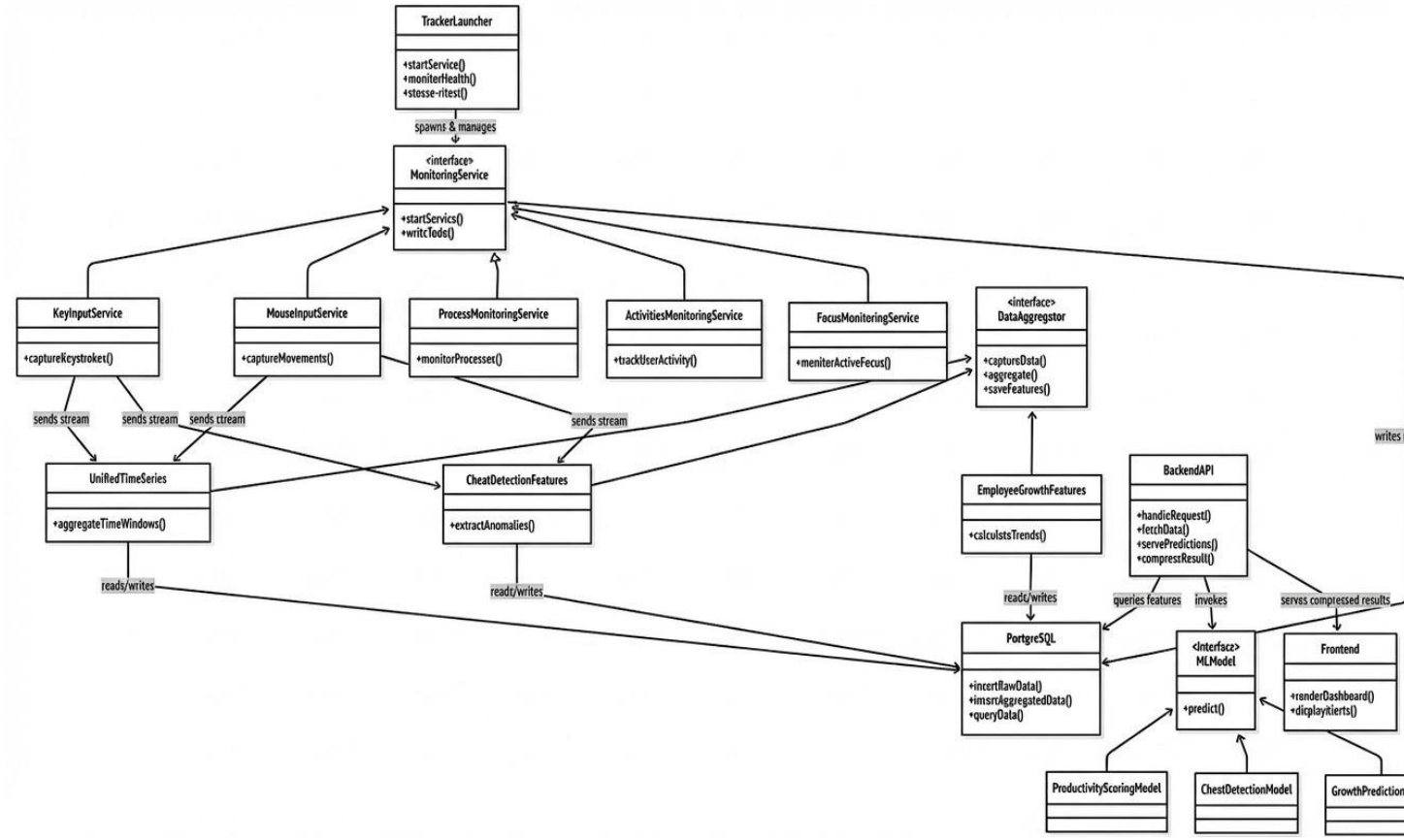
- **Data Engineering Pipelines:** Operates via SQL functions triggered by Rust. For example, populate_cheat_detection_features() aggregates events into 5-second micro-windows (321 features) to accurately detect robotic typists and mouse jigglers.
- **Growth Analytics:** The populate_employee_growth_features() function runs daily, analyzing 30-day trends to map out skill gaps, burnout risks, and overall engagement for administrative review.
- **Admin Delivery:** The backend API exposes endpoints (e.g., /api/v1/admin/cheat-alerts) that securely deliver these aggregated flags and productivity scores to the Administration Web Dashboard.

C.4.4 Data Dictionary *(A subset of the 1,556 columns highlighting core domain tables)*

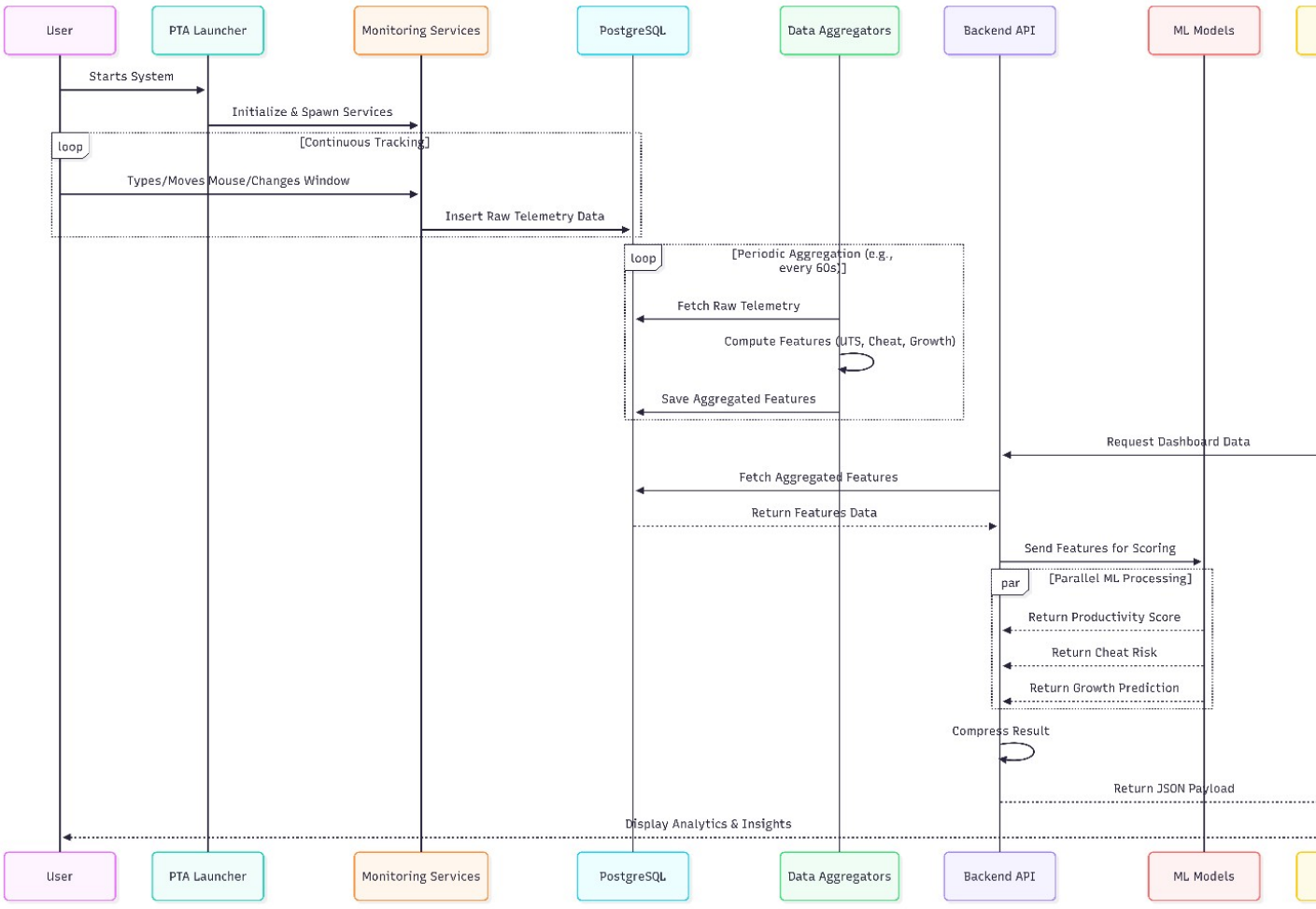
Table	Column	Type	Description
process_events	process_id	INTEGER	Windows PID
	anomaly_score	DOUBLE PRECISION	Behavioral anomaly score (0–1)
	cpu_usage_peak	DOUBLE PRECISION	Peak CPU % during scan
process_resource	memory_usage_current	BIGINT	Current RSS in bytes
	context_switches_total	BIGINT	Total OS context switches
mouse_events	Velocity	DOUBLE PRECISION	Movement speed (px/sec)
	event_type	VARCHAR(50)	'move', 'click', 'scroll', 'drag'
key_events	input_latency_ms	INTEGER	Input processing latency
	is_shortcut	BOOLEAN	Part of a keyboard shortcut
focus_events	focus_depth_score	DOUBLE PRECISION	Depth of engagement (0–1)

C.5. System Analysis & Design Overview

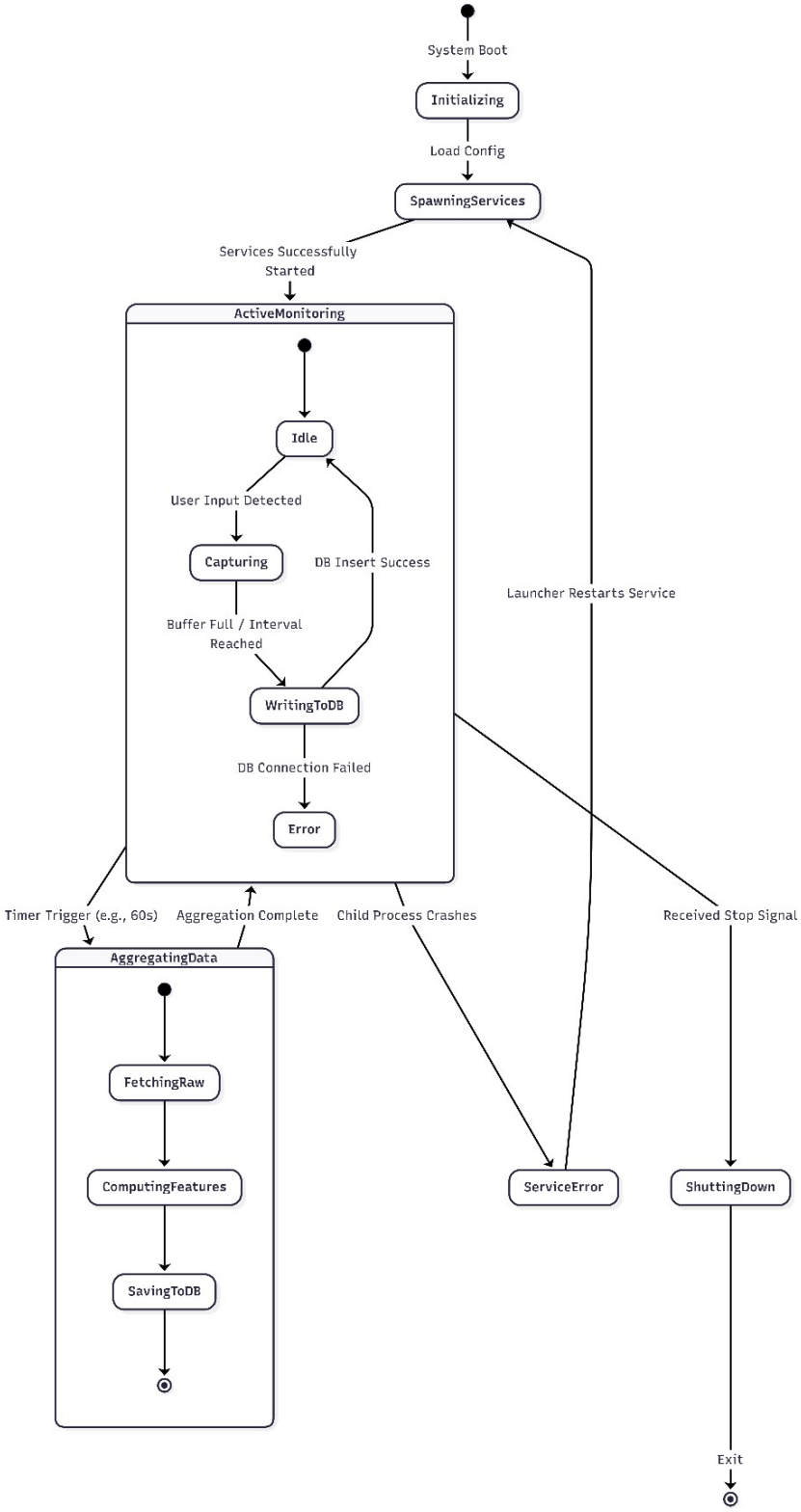
C.5.1 Class Diagrams Core Monitoring and Aggregation Architecture



C.5.2 System Sequence Diagrams Data Aggregation Sequence (Unified Time Series):



C.5.3 State Transition Diagram Process Lifecycle States:



Appendix D - Sustainable Development Goal

D.1. Introduction

The Productivity Tracking & Analytics (PTA) system is a comprehensive AI-powered platform that monitors employee desktop activity, captures multi-modal behavioral data (process execution, mouse input, keyboard input, focus patterns, and user activities), and transforms this raw data through automated data engineering pipelines into machine learning-ready features. The system aims to provide organizations with data-driven insights into employee productivity, detect integrity issues, and predict employee growth trajectories — all while promoting ethical workplace practices.

In the context of sustainable development, the PTA system directly contributes to economic productivity, decent work conditions, and responsible innovation. By enabling organizations to understand and optimize employee work patterns through AI, the system promotes sustainable economic growth, ensures fair and transparent performance evaluation, and fosters a culture of continuous professional development rather than punitive surveillance.

The system currently captures over 13 million behavioral events per working day across 27 database tables, engineers 746 ML-ready features through 3 automated pipelines, and generates actionable metrics including productivity scores, integrity assessments, and growth trajectory predictions. This data-driven approach replaces subjective performance evaluation with transparent, evidence-based insights.

D.2. SDG Alignment

D.2.1 Primary SDG **SDG 8 (Decent Work and Economic Growth):** *"Promote* sustained, inclusive and sustainable economic growth, full and *productive employment and decent work for all."*

This project directly addresses SDG 8 through the following mechanisms:

- **Target** 8.2 (Economic **productivity through technological upgrading):** The PTA system introduces AI-powered productivity analytics that enable organizations to identify productivity bottlenecks, optimize workflows, and measure real output rather than hours logged. By analyzing 746 engineered features across multiple behavioral domains, it provides granular productivity insights, representing a significant technological upgrade over legacy monitoring solutions.
- **Target 8.5 (Full and productive employment and decent work):** The system shifts the paradigm from invasive surveillance to constructive analytics. It enables fair performance evaluation by computing an overall_employee_score and providing constructive feedback paths. Furthermore, it tracks metrics like overtime_minutes and burnout_risk_composite to detect and prevent workplace exploitation, supporting accountability in remote work environments.
- **Target** 8.8 (Protect **labor rights and safe working environments):** Dedicated well-being features (avg_energy_level, avg_stress_level, fatigue_events_total) actively monitor employee health indicators, alerting management when working conditions become unsustainable to protect mental health and prevent burnout.

D.2.2 Secondary SDGs **SDG 9 (Industry, Innovation, and Infrastructure):** * **Target 9.5 (Enhance scientific research and technological capabilities):** PTA represents a significant innovation in workplace analytics via a novel data engineering architecture (Hybrid EDA + Optimized Polling in Rust). It features three automated ML pipelines running complex CTE-based SQL functions entirely inside PostgreSQL.

> **Target 9.b (Support domestic technology development):** As a locally developed FYP project, PTA demonstrates that advanced AI/data engineering solutions can be built domestically without reliance on expensive imported commercial platforms.

SDG 4 (Quality Education): * **Target 4.4 (Increase relevant skills for employment):** The system supports lifelong learning by quantifying professional development. Features like skill_breadth_30d and new_tools_adopted_30d track learning investment, while AI-generated recommendations (primary_recommendation, recommended_focus_areas) provide actionable guidance for career development.

SDG 3 (Good Health and Well-Being): * **Target 3.4 (Reduce premature mortality through prevention):** By actively monitoring indicators like consecutive_long_days, break_regularity_score, and mouse_fatigue_indicator_count, the system addresses the modern epidemic of workplace burnout and chronic stress, allowing for early intervention before mental or physical health deteriorates.

D.3. Project Impact

D.3.1 Quantitative Impact

- **Productivity Visibility:** Generation of 746 distinct ML features covering all input modalities per employee per day.
- **Data Capture Granularity:** High-frequency event capture at 163+ events per second without disrupting host performance.
- **Data Volume Capacity:** Processing capabilities of over 13,000,000+ raw events per standard 8-hour working day.
- **Burnout Prevention:** Tracking of 8 dedicated risk indicators, including a composite burnout risk score with a 7-day trending analysis.

- **Skill Development Tracking:** 15 dedicated features tracking daily learning investments, tool adoption, and skill breadth.
- **Cheating Detection Accuracy:** Utilization of 321 behavioral features for anomaly detection (e.g., typing rhythm, focus entropy) to mathematically identify artificial inputs.

D.3.2 Qualitative Impact

- **Data-Driven Performance Management:** Replaces subjective annual reviews with continuous, objective, multi-dimensional productivity measurement.
- **Ethical Monitoring Culture:** Focuses on behavioral patterns and aggregated metrics rather than invasive surveillance (no screenshots, no content capture), promoting employeremployee trust.
- **Early Intervention:** Growth trajectory predictions ('improving', 'plateaued', 'declining') enable proactive support before performance degrades.
- **Remote Work Enablement:** Provides the necessary accountability infrastructure to make remote and hybrid work sustainable and transparent for hesitant organizations.
- **Employee Empowerment:** Workers gain transparent self-assessment tools, fair recognition based on objective data, and AI-driven professional growth support.

D.4. Future Implications

D.4.1 Potential for Scalability

- **Technical Scalability:** The database schema features tenant_id structures for seamless multitenant enterprise deployment. The 6 independent tracker services can scale horizontally, and the Docker-based PostgreSQL setup is ready for cloud migration (AWS RDS, Azure).
- **Cross-Platform Expansion:** The OS-level monitoring approach can be adapted for macOS and Linux environments in future iterations.
- **Cross-Industry Applicability:** The system can scale from small freelance environments to massive enterprise deployments in any knowledge-work sector (IT, finance, legal, education).

D.4.2 Policy Recommendations

- **Establish Clear Monitoring Guidelines:** Governments and regulatory bodies should define strict frameworks regarding what behavioral data employers can collect and mandate explicit consent mechanisms.
- **Mandate Employee Data Access:** Regulations should ensure employees have the right to view their own productivity analytics, fostering a transparent workplace.
- **Adopt Ethical Internal Policies:** Organizations utilizing PTA should explicitly focus monitoring policies on productivity improvement and health support, rather than punitive surveillance.

D.4.3 Long-term Impact

- **Workplace Transformation:** The system shifts the evaluation paradigm from ”hours logged” to ”actual output behavior,” fundamentally changing how organizations compensate and reward employees.
- **Employee Well-Being & Burnout Prevention:** By detecting fatigue and stress trends early, the system can reduce burnout-related healthcare costs and lower employee turnover rates over the long term.
- **Economic Productivity Gains:** Transparent feedback loops historically increase productivity by 7-12%; applying this system at scale represents significant economic value.
- **Open-Source Architectural Reference:** The PTA system's robust 3-pipeline architecture serves as a reusable pattern for the software engineering community dealing with highvolume behavioral data.

D.5 Conclusion

The Productivity Tracking & Analytics (PTA) system demonstrates strong alignment with multiple United Nations Sustainable Development Goals, making its primary contribution to **SDG 8 (Decent Work and Economic Growth)** through AI-powered productivity analytics, fair performance evaluation, and employee well-being monitoring.

Secondary contributions to **SDG 9 (Industry, Innovation, and Infrastructure)**, **SDG 4 (Quality Education)**, and **SDG 3 (Good Health and Well-Being)** further underscore its multi-dimensional impact. By engineering 746 ML-ready features spanning productivity, integrity, growth, and wellbeing, the PTA system transcends traditional monitoring. Its ethical design principles align with the spirit of sustainable development: building systems that benefit all stakeholders while respecting human dignity, promoting continuous learning, and fostering a healthy, productive global workforce

D.6 References

- United Nations. (2015). *Transforming* our *world: The 2030 Agenda for Sustainable Development*. United Nations General Assembly. A/RES/70/1.
- International Labour Organization. (2020). *Working from Home: Estimating the worldwide potential*. ILO Policy Brief.
- World Health Organization. (2019). *Burn-out an "occupational phenomenon": International Classification of Diseases*. WHO Classification ICD-11.
- Bloom, N., Liang, J., Roberts, J., & Ying, Z. J. (2015). Does working from home work? Evidence from a Chinese experiment. *The Quarterly Journal of Economics*, 130(1), 165-218.
- PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. Retrieved from

Appendix E - Dissemination Evidence

Verifiable artefacts produced by this project:

- a publicly accessible results dashboard and methodology page served over TLS at pta.emotionflow.site;
- a public repository containing the system documentation, architecture diagrams and the figures reproduced in this report: github.com/MuhammadTaqiRahmani/Intelligent-EmployeeProductivity-Tracking-System;
- a private implementation repository holding the serving API and the machine-learning pipeline, available to examiners on request;
- a systematic review of forty papers, prepared as the research foundation for this work;
- seven technical reports covering dataset profile, data foundation, each model, consolidated results and the rule-based growth engine.

Appendix F - Marketing Material

Suggested accurate one-line description:

> PTA turns endpoint behavioural telemetry into an auditable record of how work happens — distinguishing human activity from automation at five-second resolution, and reporting capability change against each person's own baseline, without capturing a single keystroke of content.

Required disclaimer:

> PTA provides experimental model estimates of behavioural patterns. It does not measure a person's productivity, capability or intent, and must not be used as a diagnostic tool or as the basis for employment, disciplinary or security decisions without human review and independent validation.

References

1. Machine Learning Applications in Employee Turnover and Performance Prediction: A Systematic Literature Review, 2026.
2. Employees Performance Metrics Using Machine Learning: A Systematic Literature Review Using PRISMA Model, 2025.
3. Predicting Office Workers' Productivity: A Machine Learning Approach Integrating Physiological, Behavioral, and Psychological Indicators, MDPI Sensors, vol. 23, no. 24, 2023.
4. User Authentication Method Based on Keystroke Dynamics and Mouse Dynamics, MDPI Sensors, vol. 22, no. 1, 2022.
5. Employee Turnover Prediction Research, 2026.

6. Machine Learning Algorithms for Predicting Employee Performance through IoT, 2025.
7. A Survey of Deep Anomaly Detection in Multivariate Time Series: Taxonomy, Applications, and Directions, MDPI Sensors, vol. 25, no. 1, 2025.
8. AI Driven Fraud Detection Models in Financial Networks, IEEE Access, 2025.
9. Algorithmic Management on Digital Labour Platforms, 2021.
10. A SIEM-Integrated Cybersecurity Prototype for Insider Threat Anomaly Detection Using Enterprise Logs and Behavioural Biometrics, 2025.
11. Electronic Performance Monitoring in the Digital Workplace: Conceptualization, Review of Effects and Moderators, and Future Research Opportunities, Frontiers in Psychology, 2021.
12. Real-Time Detection of Insider Threats Using Behavioral Analytics and Deep Evidential Clustering, arXiv:2505.15383, 2025.
13. An Empirical Evaluation of Online Continuous Authentication Using Mouse Clickstream Data, MDPI Applied Sciences, vol. 11, no. 13, 2021.
14. Algorithmic decision-making in organizations: a systematic review toward an integrated tension alignment framework, Emerald Insight, 2025.
15. A Survey of Methods for Detecting Intentional Insider Threats Against Digital Systems, 2021.
16. Deltrux: Insider Threat Detection of End-User Behavior Analysis Using Machine Learning, IEEE, 2024.
17. Bias in AI systems: integrating formal and socio-technical approaches, Frontiers in Big Data, 2025.
18. Digital model for monitoring national programs, Frontiers in Artificial Intelligence, 2025.
19. Predicting Analytics in Human Resources Management: Evaluating AIHR's Role in Talent Retention, 2025.
20. Predicting Employee Attrition Using Machine Learning Approaches, MDPI Applied Sciences, vol. 12, no. 13, 2022.
21. Insider Threat Detection Using Behavioural Analysis through Machine Learning and Deep Learning Techniques, 2025.
22. BeCAPTCHA-Mouse: Synthetic mouse trajectories and improved bot detection, arXiv:2005.10657, 2020.
23. Employee Promotion Prediction: Machine Learning in HR Management, IEEE, 2025.
24. Predicting Employee Attrition Using Temporal Fusion Transformers (TFT), IJRCMS, 2025.
25. Application of Machine Learning in Predicting Performance and Optimizing the Recruitment Process, 2025.
26. User Behaviour Analytics in SASE: An Architectural Framework for Insider Threat Detection, 2024.
27. Employees' perceptions of the fairness of AI-based performance prediction features, Taylor & Francis, 2025.
28. A Predictive Framework for Sustainable Human Resource Management Using tNPS-Driven Machine Learning Models, MDPI Sustainability, vol. 17, no. 13, 2025.
29. The Effect of Human Resource Analytics on Organizational Performance, MDPI, vol. 13, no. 2, 2025.
30. Insider Threat Detection Using Behavioural Analysis: A Review, arXiv:2407.05943, 2025.
31. Machine learning-based approaches for enhancing human resource management using automated employee performance prediction systems, 2024.
32. Electronic Monitoring and Surveillance in the Workplace, EU JRC Publications, 2021.
33. Machine Learning Algorithms for Predicting Employee Success, 2025.
34. Effect of Electronic Performance Monitoring on Employees' Job Performance: A Social Information Processing Perspective, 2025.
35. “It's Always a Losing Game”: How Workers Understand and Resist Surveillance Technologies on the Job, arXiv:2412.06945, 2024.
36. Algorithmic Management practices in regular workplaces, ILO Report, 2021.
37. Insider Threat Agent: A Behavioral Based Zero Trust Access Control Using Machine Learning Agent, 2025.
38. Employee Performance Evaluation Using Machine Learning, 2024.

39. Analysis of Employee Performance Prediction, IJCRT, 2025.

40. Employee Performance Prediction System Using Data Mining and Machine Learning, 2025.

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that

No suspicious text manipulations found. would set it apart from a normal submission. If we notice something strange, we flag

it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Disclaimer Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.